ARC-AGI-2 Agentic Solver Portability Notes

4×L4 Offline Qwen3.8-27B + NVARC

Portable-only reference | v10 SOURCE-VERIFIED audit | 2026-09-13

Scope rule: This note keeps only architectures, algorithms, guardrails, and experiments that can be implemented or tested in the Kaggle 4xL4 offline Qwen3.8-27B system. Cloud APIs, company/funding details, vendor infrastructure, cost trivia, and non-portable deployment details are excluded.

Current target system

Component	Frozen reference
Generator A	NVARC-derived production baseline, reported 32.64 LB
Generator B	Qwen3.8-27B FP8, vLLM 0.19, TP=4 on 4×L4
Qwen profiles	OFF/non-thinking + XHIGH reasoning; same checkpoint, not multiple resident models
Primary objective	Complement NVARC candidate coverage; optimize NVARC∪Qwen oracle/pass@2, not only Qwen standalone score
Constraint	Offline execution, fixed GPU pool, strict wall-clock budget

1. Confluence Labs - Saturating ARC-AGI-2

Reference result: 97.92% Public Eval. This section keeps only the mechanics needed for a Qwen-local port.

Portable mechanics to keep

- **Program-as-candidate:** Instead of asking the LLM to memorize and write the final grid directly, require it to build `transform.py` or an equivalent executable transformation.
- **Exact training verifier:** Do not accept a generated transform unless it passes exact array equality on every training pair.
- **Failure-driven resume:** If training validation fails, feed a concise failure signal back into the same reasoning session and let it revise; do not blindly resample.
- **Independent multi-start:** Multiple independent Qwen branches should produce different hypotheses/programs; do not anchor them with a shared hint.
- **Execution is authoritative:** Take the final test grid from verified program execution, not from model prose.
- [LOCAL HARDENING, not source-exact] Confluence isolates the whole agent in an E2B sandbox, but its `test_transform()` imports and executes the generated `transform.py` inside that sandbox process rather than spawning a fresh child per transform. Our Kaggle port should add a separate child-process/process-group timeout per generated transform so an infinite loop cannot kill the notebook.
- **Per-test isolation:** For multi-test tasks, keep an independent branch per test input in the initial implementation; ablate cross-test coupling later.
- [SOURCE + LOCAL HARDENING] Confluence persists workspace/result artifacts and streams status, but the exact "write every finished branch/candidate immediately to a standalone JSON checkpoint" contract is our additional wall-clock resilience layer. Keep it, but do not attribute that exact checkpoint granularity to the source.
- [CORRECTED] The Confluence instruction is compact, but not heuristic-free: after requiring an executable transform and all-train validation, it explicitly asks the agent to consider object manipulation, color changes, spatial patterns, object relationships, and grid structure. Our ablation should compare this short generic heuristic list against an even more minimal prompt.

Qwen-local implementation mapping

Confluence mechanic	Our implementation	Priority
Gemini coding agent	Qwen3.8 XHIGH code branch with local Python tools	**P0**
E2B sandbox	Local subprocess/temp workspace + timeout + file whitelist	**P0**
Exact train validation	NumPy exact-grid validator	**P0**
12 independent agents	Small batched multi-start Qwen branches under one vLLM server	**P0**
Refinement loop	Resume-like compact state: hypothesis + code + verifier failure	**P1**
Majority top-2	Do not copy blindly; use provenance/diversity-aware pass@2	**P1**

Confluence mechanic	Our implementation	Priority
	selector	

Sources used for this portable section

Repository: https://github.com/confluence-labs/arc-agi-2
Agent runner: https://github.com/confluence-labs/arc-agi-2/blob/main/gemini-cli-solver/agent_runner.py
Orchestrator: https://github.com/confluence-labs/arc-agi-2/blob/main/gemini-cli-solver/orchestrator.py
Submission/aggregation: https://github.com/confluence-labs/arc-agi-2/blob/main/submission.py

2. Squeeze-Evolve - verifier-free evolutionary search

Reference result: 97.5% ARC-AGI-V2 Public Eval. The paper table reports the ARC setup as ancestor Gemini 3.1 Pro (High Thinking), N=4, K=2, T=10.

Most important portability result: ARC did not require their confidence/logprob scorer; it used answer-diversity fitness. Therefore the core idea can be ported to our Qwen vLLM 0.19 stack without a custom scoring fork.

2.1 Core loop - only the pieces we should port

- **Loop 0 - strong initialization:** Use the strongest available profile to generate N independent candidates. In our case, use the XHIGH profile of the same Qwen checkpoint.
- **Group candidates:** Form uniformly sampled K-sized groups. The ARC paper uses uniform selection.
- **Measure disagreement:** Canonicalize the final ARC output grids and measure group diversity.
- **Route by disagreement:** If candidates reach consensus, use free carry/medoid; as disagreement increases, route to a stronger Qwen profile/budget.
- **Recombine:** Let Qwen inspect parent hypotheses/outputs and create a new child candidate - not by copying or majority voting, but by resolving conflicts.
- **Update population:** In the baseline, replace with new children; in a later ablation, test an old+new deduplicated/medoid steady-state pool.
- **Repeat / stop:** Use a small fixed loop count or guarded convergence; checkpoint every loop.

2.2 Critical source-code finding: diversity routing direction

Implementation guardrail - SOURCE VERIFIED: the current Python `RoutingOrchestrator` still computes diversity fitness and then calls the generic `assign_routes()`. That generic router sends low fitness to stronger models and high fitness to cheaper/model_0 - sensible for confidence, but reversed for diversity. The later official plugin explicitly marks the corrected direction: high diversity/disagreement -> strong model; consensus -> free medoid. Our ARC port must use a dedicated `route_by_grid_diversity()` instead of reusing the generic router.

Why this matters for ARC: diversity is not a quality score here; it is a proxy for uncertainty/difficulty. Agreement on the same grid means easy/settled, while different grids indicate an unresolved mechanic.

2.3 ARC-specific diversity must be grid-native

Use	Definition for our solver
Exact cluster	Same shape + identical cell values -> same candidate family for hard consensus.
Near-duplicate diagnostic	For equal shapes, use cell-Hamming similarity; optionally add object/color signatures. Ablate this later as a routing metric, not as the initial gate.
Canonical key	Use a compact tuple/string containing dimensions plus the flattened grid; do not cluster ARC outputs using prose/reasoning similarity.
Provenance	Even for identical grids, retain distinct source profile, parent group, and rule/code trace provenance.

2.4 One checkpoint, three capability tiers

Squeeze-Evolve multi-model routing cannot be reproduced literally under our GPU budget, but the routing principle can be ported using inference profiles of the same checkpoint:

Route	Qwen profile	Use	GPU implication
Consensus	No generation	Carry medoid/exact agreed grid	Free
Low/cheap	OFF / non-thinking + short max_tokens	Near-consensus cleanup later	Same loaded TP4 model
Mid	Thinking/high or capped XHIGH	Moderate disagreement	Same loaded TP4 model
Strong	XHIGH + larger token budget / code branch	High disagreement / hard recombination	Same loaded TP4 model

Key design advantage: Multiple models do not need to reside in VRAM. Load Qwen3.8-27B once with TP4; the router changes only the prompt/profile/max_tokens/branch type.

2.5 K=2 has a useful but important consequence

[LOCAL INFERENCE] With the paper's ARC setup K=2, diversity is essentially binary: D=1 -> exact consensus; D=2 -> disagreement. Therefore a K=2 baseline does not really need three cheap/mid/strong tiers: consensus can be free, while disagreement triggers XHIGH recombination. Meaningful multi-profile routing requires K>=3 or additional difficulty features.

2.6 SQE-Q27-v0 - first experiment to implement

Goal: isolate whether the core Squeeze-Evolve mechanism helps Qwen3.8-27B. Do not inject the NVARC candidate pool into the evolution loop yet, so the complementary signal is not contaminated.

Parameter / stage	v0 choice	Reason
Population N	4	Matches reported ARC ancestor population.
Loop-0 generator	4 independent XHIGH Qwen candidates	Initialization quality is high-leverage; keep strongest profile here.
Candidate payload	grid + compact rule/hypothesis + optional code + origin	Allows exact clustering and later trace-aware selection.
Group size K	2	Faithful ARC baseline; gives clean consensus vs disagreement test.
Groups M	4 in first implementation	Enough recombination opportunities while bounded; treat as our implementation choice, not paper-exact claim.
Selection	Uniform	Matches ARC setup and avoids unvalidated confidence scorer.
Fitness	Exact-grid diversity	No custom vLLM logprob scorer needed.
Routing	same grid -> free carry; different -> XHIGH recombine	Correct high-diversity -> strong direction.
Update	Replace	Paper-style clean baseline.
Total loops	T=2 first (loop0 + one evolve)	Compute-aware ablation before considering deeper T.
Parser	Tolerant grid parser; do not require brittle strict JSON	Matches our prior prompt-format finding.
Checkpoint	After loop0 and every evolve loop	12h safety / reproducibility.

2.7 Recombination prompt behavior to preserve

- Keep the original ARC training examples and target test input visible at all times; the child should not solve from parent prose alone.
- Present parent candidates as possibly wrong evidence; instruct the model to resolve conflicts explicitly rather than blindly following the majority.
- Alongside parent final grids, provide compact rule/code provenance - especially when the grids disagree.
- Use a tolerant output parser; only admit a candidate into the population after its final grid has been canonicalized.
- For a code branch, execute and verify the child program on the training pairs before adding its test-grid output as a candidate.

2.8 SQE-Q27-v1 - only if v0 shows union/oracle gain

- K=3 pilot: Use the cluster ratio `n_clusters / K` to distinguish consensus, medium disagreement, and high disagreement.
- Route medium disagreement -> OFF/short-thinking; high disagreement -> XHIGH/long or code-agent branch |
- Test grid-cluster medoid deduplication plus a cap of N over the `old + children` pool; this is the later official plugin pattern for resisting duplicate collapse.

- Guarded early-stop: Even if the population collapses to one exact cluster, do not stop unless there are at least two independent origins or agreement from train-verified code.
- Do not inject NVARC candidates into the evolution loop initially; add them at the final union stage, then ablate hybrid seeding separately.

2.9 Acceptance metrics - what decides whether we keep it

Metric	Decision use
Qwen standalone pass@1/pass@2	Basic sanity only; not the primary objective.
Unique exact grids / duplicate ratio by loop	Must show evolution is not collapsing into redundant samples.
GT-in-Qwen-pool / oracle coverage	Measures generation coverage before selector effects.
NVARC ∪ Qwen oracle union gain	Primary value metric: does SQE produce answers NVARC never generated?
GT-present-but-not-selected	Separates generator improvement from future selector work.
Runtime + generations per solved incremental task	Must fit offline 4×L4 wall-clock budget.
Gain by EDA segment	Track Tier-2/3, dynamic, high-ratio, NVARC-high-duplicate and near-miss subsets.

2.10 Source links used for the port

Paper: https://arxiv.org/abs/2604.07725
Project page: https://squeeze-evolve.github.io/
Repository: https://github.com/squeeze-evolve/squeeze-evolve
Python operators:
https://github.com/squeeze-evolve/squeeze-evolve/blob/main/src/squeeze_evolve/algorithm/operators.py
Python orchestrator:
https://github.com/squeeze-evolve/squeeze-evolve/blob/main/src/squeeze_evolve/algorithm/orchestrator.py
Local/OpenAI-compatible backend:
https://github.com/squeeze-evolve/squeeze-evolve/blob/main/src/squeeze_evolve/core/backend.py
Later corrected diversity router:
https://github.com/squeeze-evolve/squeeze-evolve/blob/main/plugins/opencode-squeeze-evolve/src/core.ts
Routing unit tests: https://github.com/squeeze-evolve/squeeze-evolve/blob/main/plugins/opencode-squeeze-evolve/test/core.test.ts

3. Athanor - code-verified hypothesis refinement

Reference result: 113.83/119 = 95.7% ARC-AGI-2 Public Eval in the released audit, with one of the 120 public-eval tasks (`b6f77b65`) excluded for a documented data-quality issue. The release is a development upper-envelope of selected checkpoints rather than one uniform single-seed rerun. This section keeps only mechanics that can be ported and tested directly on our 4xL4 offline Qwen3.8-27B + NVARC system.

Main portability insight: Athanor's most useful component is not another parallel generator. It is the combination of stateful Python experimentation around Qwen reasoning, exact training verification, fresh-

context artifact-only review, and compact state handoff after rejection. All of these can be tested sequentially with the same locally loaded Qwen checkpoint.

3.1 Portable core: exploration code ≠ final solution code

- Exploratory scratchpad: Use a `run_code`-style tool for small Python experiments that deterministically check object counts, connectivity, symmetry, color mappings, coordinates, invariants, and similar properties. This is not a candidate submission.
- Stateful scratch runtime: Reuse helpers and variables from earlier exploratory calls, avoiding repeated regeneration of the same analysis code.
- Formal hypothesis artifact: Before the final program, require the model to externalize a compact natural-language transformation hypothesis.
- Formal `solve(grid)` artifact: Convert the hypothesis into a complete self-contained program; it is not a final candidate unless it passes exact verification on every training pair.
- Test outputs must come from the executable artifact; do not transcribe the grid from prose.
- Keep tool stdout canonical and compact; raw NumPy dumps or huge logs should not consume context.
- Run both exploratory code and final solve-code with a hard timeout in an isolated subprocess; do not copy Athanor's unsafe in-process execution pattern.

3.2 ATH-Q27 local loop

Stage	Our 4×L4 implementation	Acceptance condition
1. Observe / hypothesize	Qwen XHIGH gets ARC text grid + current compact state; may call stateful Python scratchpad.	At least one explicit, testable rule hypothesis.
2. Experiment	Run short local Python checks; preserve only result + verified invariant, not full transcript.	Hypothesis supported/refuted by executable evidence.
3. Submit hypothesis	Store concise rule, assumptions, ambiguity axis, expected test behavior.	Artifact is self-contained enough for a fresh reviewer.
4. Submit solve()	Complete Python `solve(grid)`; local harness executes on ALL train pairs.	Exact train pass; malformed/duplicate outputs rejected.
5. Self-audit	Cheap short call checks hard-coded coordinates/sizes/colors and test-output plausibility.	ACCEPT or RETRY.
6. Artifact-only review	Fresh Qwen context receives puzzle + hypothesis + solve() + train result + predicted test grids; no solver transcript.	APPROVE / REJECT / EXPAND_CANDIDATES.
7. Act on verdict	APPROVE->candidate pool; REJECT->fresh context + compact handoff; EXPAND->justify second candidate.	No stale-context retry after hard rejection.

3.3 Independent reviewer - port the information boundary, not the cloud model

Key adaptation: Original Athanor can use a separate model/context for the reviewer. We will keep a single resident TP4 Qwen3.8-27B, so independence initially comes from context isolation rather than model family:

fresh request, zero solver transcript, artifact-only evidence. Solver and reviewer run sequentially; do not load two 27B instances simultaneously on the same four GPUs.

- Have the reviewer inspect the puzzle's train/test inputs and sketch its own rule before reading the solver hypothesis/code; retain this ordering to reduce confirmation bias.
- Run the reviewer only on candidates that pass 100% of training examples; do not spend reviewer compute on weak candidates.
- Reviewer checks: Does the hypothesis truly explain every training pair? Does the code faithfully implement the hypothesis? Are there training-only constants or hard-coded special cases? Are the test predictions plausible consequences of the same rule?
- Keep verdict semantics strict: REJECT = the latent rule/code artifact is wrong or overfit; EXPAND_CANDIDATES = the rule is sound but the training data cannot resolve a concrete ambiguity; APPROVE = the candidate set is sufficient.
- Reviewer feedback is advisory: the solver should not blindly obey the reviewer and may challenge a critique with concrete training evidence.
- Disable the reviewer's code-execution tool in the first prototype; if reviewer value is demonstrated, later ablate a version in which the reviewer can execute alternative hypotheses.

3.4 EXPAND_CANDIDATES is directly aligned with ARC pass@2

Do not use attempt-2 as generic rank-2. Athanor's portable idea is to open the second slot only when a specific ambiguity axis produces two train-consistent rules that yield different test grids. The reviewer identifies the missing behavior; the solver then generates a justified second candidate from the same rule family.

Integration with our NVARC finding: This directly addresses the current NVARC attempt-2 weakness. The second slot should be a mechanic-diverse, puzzle-grounded alternative rather than simply the next answer from the same ranker.

3.5 ICAE-style compact state reset for Qwen ~30K practical context

- Do not carry the raw full conversation indefinitely. Start a fresh Qwen context under context pressure or after a reviewer REJECT verdict.
- At handoff, provide the original puzzle data again, followed only by portable research state; do not transfer a chain-of-thought transcript.
- After REJECT, combine reviewer critique with state compression/reset in the same transition so the model does not keep circling inside stale context around the rejected hypothesis.
- Do not copy Athanor's 170K threshold. Trigger from the actual Qwen notebook `MAX_MODEL_LEN` with safety headroom (initial pilot: about 65-75% of usable context, tuned with measured token counts).
- Even if the compression summary is model-generated, enforce deterministic fields and preserve critical code/results separately as verbatim artifacts.

Proposed compact research-state schema:

- current_hypothesis
- verified_invariants
- disproven_hypotheses
- key_experiments: {purpose, code_hash/snippet, compact_result}
- current_solve_code
- train_failure_signature / exact train status
- predicted_test_grids

- reviewer_feedback
- next_action

3.6 Context/output guardrails worth copying

Guardrail	Our implementation
Tool output truncation	Long stdout/error capped; save full raw artifact to disk, feed model only compact tail/summary.
No hidden test leakage	Exploratory runtime exposes train input+output and test input only.
Candidate normalization	Accept only rectangular integer grids 0-9; max two; exact duplicate candidates collapse.
Hard timeout	Subprocess/process-group kill; one bad generated loop cannot hang Kaggle run.
Checkpoint after formal attempt	Persist hypothesis, code, train metrics, predictions, reviewer verdict immediately.
Reviewer failure is non-fatal	If reviewer parse/call fails, retain train-verified candidate with lower confidence; do not lose generator output.

3.7 Athanor experiment ladder - isolate value one mechanism at a time

Experiment	Add	Keep fixed	Primary metric
ATH-Q27-A0	Stateful exploratory Python + formal solve() exact-train verifier	Same Qwen prompt/profile/budget as direct code baseline	GT-in-Qwen-pool; train-pass rate; runtime
ATH-Q27-A1	Explicit hypothesis artifact + fresh-context reviewer, APPROVE/REJECT only	A0 generator unchanged	Wrong train-perfect detection; rescue after reject; false-reject rate
ATH-Q27-A2	EXPAND_CANDIDATES verdict + justified second output	Reviewer otherwise unchanged	Pass@2 gain; second-slot unique solves
ATH-Q27-A3	ICAE reset on reject/context pressure	A2 logic unchanged	Recovery after reset; context tokens; repeated-dead-end rate
ATH-Q27-A4	Union with NVARC pool; provenance-aware selection	Generators frozen	NVARC∪Qwen oracle union; GT-present-but-not-selected

3.8 Reviewer-specific metrics - required before keeping it

- False reject rate: how often the reviewer REJECTs known-correct/public-GT Qwen candidates.
- Rescue rate: after the first train-perfect but GT-wrong candidate is REJECTed, how often a later attempt generates a GT candidate.
- Overfit catch rate: fraction of 100%-train-pass but test-GT-wrong candidates caught by the reviewer.
- EXPAND precision: whether the second candidate truly covers a distinct rule ambiguity rather than acting as a random hedge.
- Incremental union gain: number of new GT candidates added by the reviewer loop beyond the NVARC baseline.

- Wall-clock / token budget: whether reviewer calls plus retries violate the per-task compute budget.

3.9 What we deliberately do NOT port

- Cloud provider SDK, prompt caching, Phoenix/Web UI, vendor-specific thinking API.
- 170K compression threshold as a literal constant.
- Unsafe in-process model-generated Python execution.
- Vision/image path in first Qwen experiment (`language_model_only=True` remains baseline).
- Long generic ARC-domain prompt as-is; retain only tool contract + concise solving objective.
- “Best effort, ignore train accuracy” escape hatch in first implementation.
- Different-model-family reviewer requirement; first test is same Qwen weights with strict fresh-context separation.

3.10 Validation caveat that changes how we benchmark the port

Controlled evaluation required: Athanor reports 113.83/119 = 95.7% on the 120-task public eval with `b6f77b65` excluded for a documented data-quality issue. The released 119 checkpoints are an auditable development upper-envelope: for each puzzle the best-scoring run was selected (lowest cost on ties); 89/119 checkpoints use the final prompt and 30 use earlier prompt drafts. This is not one uniform single-seed run. Therefore ATH-Q27 ablations must use one frozen protocol/budget on the complete proxy/public slice rather than compare a single run directly to the 95.7% headline.

3.11 Source links used for the port

Repository: https://github.com/dastin359/athanor
Design rationale: https://github.com/dastin359/athanor/blob/main/docs/design.md
Solver tool contract: https://github.com/dastin359/athanor/blob/main/src/athanor/solver/SOLVER_SYSTEM_PROMPT.md
Canonical orchestrator: https://github.com/dastin359/athanor/blob/main/src/athanor/solver/orchestrator.py
Independent reviewer: https://github.com/dastin359/athanor/blob/main/src/athanor/solver/independent_reflector.py
Reviewer contract: https://github.com/dastin359/athanor/blob/main/src/athanor/solver/REFLECTOR_SYSTEM_PROMPT.md
Compression/reflection prompts: https://github.com/dastin359/athanor/blob/main/src/athanor/solver/prompts.py
Released result methodology/traces: https://github.com/dastin359/athanor/blob/main/RESULTS.md

4. Imbue - Darwinian Evolver: population-level code evolution

Reference result: 95.1% ARC-AGI-2 Public Eval with Gemini 3.1 Pro. The result number is retained only for provenance; implementation focus is entirely on mechanics that can be ported to offline Qwen3.8-27B + NVARC.

Why this one is different: Confluence uses independent multi-start + voting; Squeeze-Evolve evolves responses/candidates; Athanor uses single-thread code verification + independent review. Imbue's unique portable idea is a persistent global population of executable ARC programs combining fitness-weighted parent selection, targeted mutation, crossover, and lineage/archive search.

4.1 Exact ARC loop worth porting

Stage	Exact source behavior	Qwen-local port
1. Seed	Population starts from a simple executable transform; every organism is code plus an explanation.	Start from identity or our direct-Qwen verified program seeds.
2. Evaluate	Run candidate code on all train examples; exact matches + per-cell soft accuracy feed fitness.	Use isolated subprocess, exact-grid validator and compact diff feedback.
3. Select parents	Weighted sampling from the full archive, not just greedy best-so-far.	Sample by fitness sigmoid x novelty bonus.
4. Mutate	LLM revises a selected parent using its code, explanation, train feedback and visible test inputs/predictions.	One resident Qwen sequentially produces child programs.
5. Verify mutation	ARC implementation rejects mutations that do not change output on any train/test input.	Cheap semantic-change gate before full scoring.
6. Archive	Accepted evaluated children are added; population update is atomic per iteration.	Keep lineage, score, outputs and provenance for every child.
7. Crossover	Occasional 3-parent synthesis combines discoveries from separate lineages.	Late-stage ablation only after population has useful diversity.
8. Stop	Do not stop on first train-perfect program; use transfer + population agreement/diversity evidence.	Budget-aware stop gate calibrated for Qwen.
9. Pass@2	Attempt 1 from best program; attempt 2 is highest-scoring different output per test input.	Directly reusable for NVARC/Qwen candidate union.

4.2 Parent selection: exploit strong lineages without collapsing diversity

- WeightedSamplingPopulation uses weight = sigmoid_performance x novelty_bonus. The score midpoint is dynamic, so relative population quality matters rather than a fixed absolute cut.
- Exact ARC reproduction config uses sharpness=10, midpoint_score_percentile=99 and novelty_weight=0.2. The high percentile strongly exploits top programs, while novelty bonus still reduces repeated expansion of already-explored parents.
- Novelty bonus is 1 / (1 + novelty_weight * number_of_children). Therefore even a strong parent gradually becomes less dominant as many children are tried.
- All eligible parents keep positive probability; a non-best lineage can re-enter later and escape a local optimum.
- For Qwen, treat p99 as an ablation, not gospel: start with p90 vs p99 on the same hard subset and compare unique-GT discovery per model call.

4.3 Mutation prompt: the highest-value portable part

- Each organism carries both executable code and a natural-language explanation of the transformation. Mutation prompt asks for transformation steps first, then a complete revised program.

- Mutator receives all training examples, challenge/test inputs, parent explanation/code and detailed evaluation feedback. This lets Qwen repair a concrete failure instead of restarting from a blank prompt.
- For same-shape/high-similarity tasks, input/output differences are rendered explicitly so changed cells are easy to inspect; failed predictions also show prediction/correct cell diffs and per-example soft accuracy.
- Mutation strength is randomized: roughly half the time it asks for a local adaptation/repair; the other branch asks to reconsider the rule more aggressively. This creates structured exploration without needing another model.
- If a parent already solves every training example, prompt objective changes from train fitting to generalization on visible challenge inputs while preserving the training rule.
- Test predictions are shown back to the mutator without pretending they are correct. This is useful for detecting visually implausible extrapolations.

4.4 Fitness structure: keep the hierarchy, recalibrate the judge

Component	Exact ARC weight	What source does	Our port decision
Correctness	90%	Mean per-cell train score, rescaled so trivial identity baseline maps to 0.2; exact train-perfect gets a bonus and reaches 1.0.	P0. Reproduce with our verifier; exact train pass remains dominant.
Transfer / generalization	7%	Only after exact train pass, a reasoning judge scores whether explanation/rule applies to challenge inputs, is correctly applied, and stays consistent with examples; the three subscores are multiplied.	P1. Use a fresh Qwen context, then calibrate threshold on public-GT tasks. Never copy Gemini threshold 0.95 literally.
Simplicity	3%	Auxiliary code heuristic/tie-breaker based on literals, hard-coded colors and code length.	Do not copy exact formula initially; use deterministic penalties or ablate entirely until validated.

Important calibration rule: Exact code uses provider-specific transfer thresholds (Gemini 0.95, OpenAI 0.70, Anthropic 0.35). That is direct evidence this score is model-calibrated. Qwen must get its own threshold from public-GT calibration; 0.95 is not a portable constant.

4.5 Source-code findings that prevent a wrong port

- **Generic batch mutation is not the ARC bottleneck:** Exact ARC mutator ignores the sampled failure-case mini-batch for prompt construction and rebuilds feedback from the parent full evaluation: all train results plus challenge predictions. `batch_size=32` is therefore not a first-port requirement.
- **ARC verification is change, not improvement:** The exact `verify_mutation` executes parent and child across all train + test inputs and accepts the child when at least one output changes. It is a no-op rewrite filter, not an "improved failed example" gate.
- **Stopping rule is stricter than the headline prose:** Exact reproduction commit requires a train-perfect/high-transfer solution plus population agreement/diversity conditions; it does not simply stop at the first or even arbitrary two train-perfect programs.

- **Learning log is optional for the reported ARC run:** Framework supports lineage learning logs, but ARC CLI default is `learning_log=none`. Do not build that infrastructure before the basic evolver proves value.
- **Exact simplicity formula deserves an ablation:** Current source computes a final simplicity term that includes `1 - length_score`, which can reward longer code despite the stated simplicity goal, and branch score is not included in the final combination. Copying this 3% term verbatim is unsafe.
- **Attempt 2 is per test input:** The second ARC slot is chosen independently for each test case by scanning score-sorted population for the best output different from attempt 1. One second program does not need to explain every test input.

4.6 Crossover: useful only after the archive has real discoveries

- ARC crossover is enabled with frequency 0.25, minimum population size 3 and three parents per crossover.
- Crossover parent sampling uses a stronger novelty preference than ordinary mutation; the prompt receives each parent explanation, code and evaluation feedback and asks for one unified transform.
- Our Qwen version can do this sequentially with the same resident model; no multi-model or extra GPU instance is required.
- Gate crossover children exactly like normal children, then measure whether crossover adds new GT-containing output clusters. If it mostly collapses diversity or consumes calls, remove it.

4.7 Pass@2 selection is especially valuable for our NVARC union

Portable rule: For each test input, attempt_1 = prediction from the highest-ranked candidate. attempt_2 = highest-ranked canonical output that differs from attempt_1. Keep the originating program/provenance with each grid, but do not force all test examples to use the same second organism.

- Dedupe by canonical grid before selecting attempt 2; textual/code diversity without output diversity gives no ARC value.
- When Qwen and NVARC pools are merged, allow attempt 2 to come from a different generator family/lineage if it covers a distinct output cluster.
- Track `GT present but not selected`: if oracle pool contains the truth but final two slots miss it, improve ranking/selection rather than generator compute.
- Compare highest-score-different vs provenance-diverse-different vs ambiguity/reviewer-guided second slot on exactly the same frozen candidate pool.

4.8 IMB-Q27 experiment ladder - bounded for 4xL4

Experiment	Add	Budget discipline	Primary metric
IMB-Q27-I0	Small global archive + normal/aggressive parent mutation + exact/soft correctness fitness + changed-output gate	1 parent/round, 4-6 rounds, no transfer judge/crossover	Unique GT candidates per Qwen call; best train score; pool oracle
IMB-Q27-I1	Weighted parent sampling with sigmoid + novelty; compare p90 vs p99	2 parents/round only on hard subset	Oracle gain per call; lineage depth; diversity
IMB-Q27-I2	Differential train feedback + train-perfect generalization	Same I1 call budget	Train-perfect-to-test-correct conversion; repeated-dead-

Experiment	Add	Budget discipline	Primary metric
	mutation route		end rate
IMB-Q27-I3	Fresh-context Qwen transfer judge for train-perfect programs	Calibrate threshold from public GT before use as stop/rank signal	Judge AUC/precision; false-confidence rate; saved calls
IMB-Q27-I4	3-parent crossover at 25%	Only if population >=3 and I0-I3 show useful lineages	Crossover yield; new output clusters; GT rescue
IMB-Q27-I5	Seed/archive union from Direct + Athanor + SQE + NVARC-derived program candidates	No new generator until selection study finished	NVARC union oracle; final pass@2; GT-present-not-selected

Compute warning: The reference default can run up to 16 evolution iterations and multiple mutations/crossovers per iteration. On one TP=4 Qwen3.8-27B server this can become dozens of generations per hard task. Therefore do not copy the full reference budget; measure mechanism value on a small hard-subset pilot with strict generation/time caps.

4.9 Metrics required before keeping Darwinian evolution

- Useful mutation rate: how often a child improves the parent's training soft/exact score.
- No-op rejection rate: how many textual rewrites are discarded by the semantic-change gate.
- Unique output clusters and lineage depth per task.
- Best fitness / train accuracy versus iteration and model calls.
- Count of train-perfect but mutually different challenge predictions.
- Transfer-judge calibration on public GT: AUC/precision/false-approve for train-perfect candidates.
- Crossover yield: whether crossover children produce useful/GT outputs not already present in the mutation-only pool.
- Attempt-2 unique solves and GT-present-but-not-selected rate.
- NVARC union oracle gain, final pass@2 gain, wall-clock and generated tokens per task.

4.10 What we deliberately do NOT port

- Google/OpenAI/Anthropic/OpenRouter provider routing, API retries, cost trackers or cloud-specific code execution.
- 32-task concurrency and 5-hour-per-task defaults; our TP4 Qwen is one shared resident inference server.
- Gemini-specific transfer threshold 0.95 or any provider-specific thinking setting.
- The exact 3% simplicity formula until it proves useful under Qwen.
- Generic learning-log/batch infrastructure in v0; exact ARC reproduction does not require it by default.
- Unbounded 16-iteration search across the full set before a bounded hard-subset ablation proves incremental union gain.

4.11 Source links used for the port

Imbue ARC-AGI-2 report: https://imbue.com/blog/2026-02-27-arc-agi-2-evolution
Darwinian Evolver method report: https://imbue.com/blog/2026-02-27-darwinian-evolver
Repository: https://github.com/imbue-ai/darwinian_evolver

Exact 95.1 reproduction commit:
https://github.com/imbue-ai/darwinian_evolver/tree/1a50d6995e893580c4bd5071725eb24f2317d380
ARC runner at reproduction commit:
https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/imbue_experiments/run_arc.py
ARC evaluator/mutators:
https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/problems/arc_agi.py
Population / weighted sampling:
https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/population.py
Evolution loop:
https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/evolver.py
ARC diff feedback / two-attempt builder:
https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/problems/arc_agi_poetiq.py

5. Noemon - test-blind rule learning + fresh-context validation

Reference-result label requires a caveat: the Noemon repository reports 92.5% as ARC-AGI-2 Public Eval, while the current ARC Prize Community Leaderboard labels the same 92.5% entry as "ARC-AGI-2 Public Train". Because public results are self-reported and this regime label is inconsistent across the two primary surfaces, treat 92.5% as a public self-report with unresolved split labeling - not as a semi-private verified result.

5.1 Portable core loop

Stage	Noemon source behavior	Our Qwen3.8 port
1. Test-blind rule induction	Round 1 receives training pairs only and emits deterministic natural-language instructions. Test input is intentionally absent.	Qwen XHIGH forms one rule packet from train pairs only; do not expose test until rule has been challenged.
2. Independent rule application	Validator gets the instructions, non-target train examples and one held-out train input; the held-out output is hidden.	Fresh Qwen context applies the rule to a held-out train input. Compare externally to GT; use mismatch diff as feedback.
3. Feedback refinement	Reasoner keeps its conversation and revises instructions from validator failures.	Carry a compact hypothesis-state packet rather than full long XHIGH transcript.
4. Target solve	After validation / max feedback rounds, reasoner sees the test input and emits a grid.	Only now reveal test input and produce candidate grid.
5. Test-target validator	A separate validator independently applies the same instructions to the test input.	Fresh-context OFF/short or XHIGH-ablation; agreement is confidence evidence, disagreement creates an alternate candidate.
6. Final-solver variants	Extra solvers use latest instructions;	Generate 1-2 cheap independent

Stage	Noemon source behavior	Our Qwen3.8 port
	after train success each sees only one rotating train pair as reference.	variants only on unresolved/hard tasks; rotate reference pair/seed/profile.
7. Candidate selection	Exact duplicate can stop early; otherwise a judge chooses from the candidate set.	Do not trust raw duplicate alone. Require provenance-diverse agreement or send candidate-only pool to calibrated selector/judge.

5.2 Highest-value idea: hide the test input while learning the rule

- `reasoner_round_1` serializes training pairs but explicitly omits the test input; round 2 receives validator feedback and any overflow training pairs, still without target anchoring.
- Only round 3 introduces the test input. This cleanly separates "infer a general transformation" from "apply it to this target".
- For our Qwen system this is a strong anti-overfit ablation: compare current direct prompt versus train-only rule induction -> validation -> target reveal on the same frozen tasks.
- Persist a compact rule packet: high-level rule, output-size rule, invariants, transformation algorithm, rejected alternative(s), latest validator failures. Do not persist unrestricted chain-of-thought.

Why it matters for NVARC + Qwen: NVARC already specializes in direct grid generation. A test-blind Qwen rule branch is more likely to contribute a different hypothesis family rather than imitate another direct decoder.

5.3 Validator = leave-one-out generalization test for natural-language rules

- For a target training pair, validator sees the rule instructions and other training examples, but the target output is hidden. It must reconstruct the target output from the rule.
- Validation is therefore stronger than asking the reasoner whether its own rule "looks correct"; a fresh context must operationalize the rule on unseen-within-task data.
- The source uses a cheap two-example probe first. If both probe examples pass, it validates the remaining unsolved train examples; if probe fails, it stops early and returns feedback.
- Failure feedback is concrete: invalid grid, shape mismatch, or exact cell-diff grid. Our port should preserve this structured evidence because it gives Qwen actionable repair targets without another long analysis prompt.
- Validator context is independently shuffled and capped by a grid-cell budget. For Qwen, calibrate the budget from actual tokenizer counts; do not copy Gemini cell limits literally.

5.4 Source-code trap: do NOT carry "solved train indices" across changed rules

Important source finding: The current Noemon batch engine excludes training indices that were solved in prior validation rounds. If every index has been solved at some earlier point, it can mark the current state train_correct=True without re-running the current instructions on all training pairs. Because the instructions may have changed between rounds, this is not a logically valid proof that the current rule solves every train pair.

- Our rule: cache a validator result only while `hypothesis_hash` is unchanged.
- Whenever Qwen revises the instructions, invalidate prior per-example passes for acceptance purposes.
- Before a rule is allowed to trigger "train verified", run a final full validation sweep of the CURRENT instructions across every training pair (or exact execution for a code candidate).

- Probe validation may save compute during search, but it cannot replace the final current-hypothesis full check.

5.5 Test-target agreement is a useful confidence signal - but not proof

- After train validation, Noemon has another fresh validator apply the same instructions to the actual test input.
- If reasoner grid and validator grid differ, the validator prediction is preserved as another candidate and the mismatch/shape feedback can be returned to the next reasoning step.
- Our first measurement should be `P(correct | reasoner == validator)` versus `P(correct | disagree)` on public GT. Only after calibration should agreement influence early stopping.
- Use different Qwen profiles/context/order for reasoner and validator so agreement is less correlated: e.g. reasoner XHIGH, validator OFF/short first; XHIGH validator only as an ablation on hard rules.

5.6 Self-consistency: keep the signal, replace the stopping rule

Do not copy raw duplicate-stop: Source accepts an exact grid once it has appeared twice. With one model family, duplicate answers can be shared bias/groupthink. For us, exact agreement becomes strong only when the two copies have meaningfully different provenance.

- Strong consensus: Qwen rule-validator agrees with Qwen code-agent, or Qwen agrees with NVARC, or independently conditioned Qwen branches agree after current-rule train verification.
- Weak consensus: same prompt family / same instructions / same reference pair repeatedly samples the same grid. Do not spend an ARC attempt slot or stop search just because of this.
- Log consensus provenance explicitly so we can measure false-consensus rate and determine which combinations are actually trustworthy.

5.7 Final-solver diversity with one rotating reference pair

- When a rule has passed training, the final solver is intentionally lightweight: latest instructions + test input + one training pair for reference.
- The chosen train pair rotates across final-solver attempts, avoiding the same reference pair consecutively when possible. This changes conditioning cheaply without discarding the learned rule.
- For our 4xL4 budget, do not run all three reference final-solvers on every puzzle. Start with one; add a second only if rule/test-validator disagreement or NVARC disagreement remains.
- If training is not fully validated, final solver should receive the relevant failure feedback and a larger train subset instead of pretending confidence.

5.8 Context and throughput tricks worth porting

Mechanic	Why useful	Qwen adaptation
Randomized train order	Different independent attempts see a different order; helps diversity without new models.	Use fixed recorded seeds per experiment for reproducibility.
Progressive disclosure	If round-1 train budget overflows, unseen train pairs are revealed in round 2.	Useful for large ARC examples under our ~30K practical context.
Compact grid serialization	Rows stay compact but line-separated; optional shape + color counts are deterministic metadata.	Ablate metadata; keep only if Qwen accuracy or token efficiency improves.

Mechanic	Why useful	Qwen adaptation
Stage checkpointing	State stores instructions, candidates, validation, feedback and decisions after each stage.	Use lightweight local JSON; makes 12h Kaggle run resumable/debuggable.
Stage-wave execution	Cloud implementation batches the same stage across tasks.	Translate to local vLLM waves: reasoner batch -> validator batch -> refinement/solve batch -> judge only unresolved tasks.

5.9 One resident Qwen model, four logical roles

Role	Initial local profile	Purpose
Reasoner	Qwen XHIGH; moderate sampling diversity; train-only context	Infer rule/invariants, revise from evidence.
Validator	Fresh context; OFF/short first, XHIGH as hard-case ablation	Apply supplied rule, not invent a new one; expose generalization failures cheaply.
Final solver	Same model sequentially; different seed/reference pair and optionally OFF vs XHIGH	Generate independent target grids conditioned on the validated rule.
Judge / selector	Fresh XHIGH, low randomness, candidate-only evidence	Choose among a small canonical grid pool; treat prior instructions as hint, not truth.

GPU constraint: These roles are not four 27B models running at once. Use the same TP=4 Qwen3.8-27B instance for all logical roles through sequential/batched calls, so no extra model VRAM is required.

5.10 NOE-Q27 experiment ladder - isolate each contribution

Experiment	Add	Budget discipline	Primary metric
NOE-Q27-N0	Train-only instruction rule -> FINAL full current-rule train validation -> one target solve	One reasoner refinement max; no test-validator, duplicate stop, final-solver ensemble or judge	Standalone accuracy; NVARC union oracle; rule train-pass rate
NOE-Q27-N1	Two-train probe before expensive full validation; invalidate cache whenever rule changes	Same N0 generation budget	Validator calls saved; probe false-pass/false-fail vs final full sweep
NOE-Q27-N2	Fresh test-target validator; disagreement adds alternate grid + compact diff	Run only after current-rule train pass	P(correct|agree/disagree); unique GT rescue; extra calls
NOE-Q27-N3	1-2 final-solver variants with rotating single train reference / seed / mode	Only unresolved or NVARC-disagreement tasks	New output clusters and oracle gain per call
NOE-Q27-N4	Provenance-diverse consensus + small candidate-	No full hidden reasoning transcript to judge	Early-stop precision; judge recovery/corruption; final

Experiment	Add	Budget discipline	Primary metric
	only judge		pass@2
NOE-Q27-N5	Local stage-wave vLLM scheduler + adaptive second independent attempt	Second full attempt only on low-consensus/hard tasks	Wall-clock, GPU utilization, solved/task-hour, LB proxy

5.11 Metrics required before keeping Noemon mechanics

- Rule generalization: current instruction packet full-train pass rate and target accuracy.
- Probe validator: calls saved, false-pass and false-fail relative to final all-train validation.
- Test-validator calibration: P(correct | agree), P(correct | disagree), and correctness of the validator-only alternate grid.
- Candidate diversity: unique canonical grids, provenance clusters, NVARC+Qwen oracle union and unique GT additions.
- Consensus quality: false-consensus rate separated by provenance pair (Qwen/Qwen, Qwen-code, Qwen/NVARC).
- Judge quality: GT present in pool -> selected/not selected; judge recovery versus corruption.
- Compute: Qwen calls, generated tokens, elapsed seconds, stage-wise GPU utilization and tasks/hour.

5.12 What we deliberately do NOT port

- Gemini Batch API, remote batch registry, provider pricing/retry infrastructure or Gemini thought-signature handling.
- Gemini-specific temperatures, reasoning levels, 65K output budget, 3000/2000 grid-cell limits or exact schema behavior.
- Strict guided JSON as a mandatory Qwen interface; use a tolerant final-envelope parser and test strict-vs-tolerant as an ablation.
- Cross-round union of previously solved train indices after the rule changes.
- Unconditional "same grid appeared twice -> stop" without provenance-diverse evidence.
- Two complete six-stage attempts on every task before adaptive routing proves that the extra compute adds NVARC-complementary GTs.

5.13 Source links used for the port

Repository snapshot audited:
https://github.com/noemon/noemon-arc-agi/tree/54c7c3af9aa95a495586cba8caffc9ca526f9924
README / reported result and architecture:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/README.md
Prompt bundle:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/prompt_bundle.md
Prompt construction / train hiding / rotating reference: https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/prompts.py
State machine / candidate and test-validator logic:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/engine.py
Batch state machine / probe validation / duplicate / judge: https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/batch_engine.py

Iteration/state models:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/models.py
Runtime stage options:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/runtime_config.py
Structured response schemas:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/response_schemas.py
Prompt behavior tests:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/tests/test_noemon_prompts.py
End-to-end Noemon batch/state tests:
https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/tests/test_noemon_batch.py

6. Symbolica ARCgentica / Agentica - persistent REPL + dynamic scoped delegation

Reference result: 85.28% ARC-AGI-2 Public Eval (repository / official Symbolica self-report) with Opus 4.6 High. This section excludes provider/API/cost infrastructure and keeps only mechanics that can be ported or tested as controlled experiments in our 4xL4 offline Qwen3.8-27B + NVARC system.

Source audit scope: The audit covered the complete public solver snapshot, parent/sub-agent wrapper, prompts, typed ARC helpers, program runner, scorer, submission logic, summary code, exact final run config, Agentica SDK runtime model, committed final parent/child logs, and final result JSONs. The public repository contains a single initial snapshot commit, so committed final traces/results were used as execution evidence instead of branch archaeology.

6.1 What the portable core actually is

Mechanic	ARCgentica source behavior	Our Qwen3.8 port
Persistent Python REPL	Parent agent repeatedly runs Python, inspects objects/results, keeps runtime state and refines its hypothesis before returning a program.	Give each logical Qwen agent a persistent local Python namespace and an action loop: reason -> Python -> observation -> reason. Do not reconstruct every intermediate object in prompt text.
Program synthesis	Final artifact is a Python `transform(grid) -> grid` function plus explanation; harness then executes the same code on every train/test input.	Treat executable program as one candidate family. External exact train verifier is authoritative; model prose is not.
Dynamic delegation	Parent can spawn focused child agents only when useful; child gets a self-contained task plus selected runtime objects, not the parent's full transcript.	Use the same resident TP4 Qwen model with independent prompt/context states. Parent decides whether to spend 0/1/2 extra child calls.
Recursive delegation	Children also receive `call_agent`, so they can delegate further. Width/depth	Do NOT enable recursive grandchildren in v0. Allow depth-2 only after 1-level

	is model-controlled but capped by a total-agent budget.	delegation proves unique NVARC-complementary GT gains.
Typed result contract	Agents return `FinalSolution(transform_code, explanation)` or focused analysis strings.	Use a small tolerant artifact envelope: hypothesis, evidence, optional code, train_fit, predicted_grid, caveats. Avoid strict provider-specific tool schemas.
Exact + soft verifier	Exact whole-grid accuracy is binary; soft accuracy is cell fraction when shape matches.	Exact = final acceptance gate. Soft score = search/refinement signal only; never final submission eligibility.

6.2 Final-run logs prove delegation is uncertainty-triggered, not a fixed ensemble

- Representative committed log `0934a4d8/attempt_0`: parent first uses its own REPL to inspect the puzzle and discovers a strong D2-symmetry hypothesis. The challenge then exposes an edge case where part of the masked region has no simple in-bounds symmetry partner.
- At that unresolved point the parent constructs two different self-contained sub-tasks and launches two children concurrently with `asyncio.gather(...)`. The children independently probe the structural hypothesis in their own REPLs and return compact analyses; the parent then synthesizes the result into executable code.
- This is the part worth porting: spend extra contexts on a concrete uncertainty, not "always run 10 agents". The published final run allowed up to 9 spawned children plus the initial parent, but the reported average was only about 2.6 agents per task attempt.
- The exact final config used two independent attempts and a 5-second transform timeout. For our local system these are reference points, not constants to copy; adaptive routing should decide whether a second full branch is worth the Qwen compute.

6.3 Minimal local REPL design - no Agentica server required

Port the behavior, not the framework: our first implementation should NOT depend on Agentica Server or any cloud adapter. One resident TP=4 Qwen3.8-27B instance can service many logical agents as separate message/state objects; vLLM batches the active sequences while model weights remain loaded once.

- Logical agent state: `messages`, `scratch_summary`, `python_namespace_id`, `hypothesis_id`, `current_code`, `train_metrics`, `candidate_provenance`, `budget_used`.
- Python scope: train/test grids, NumPy, `scipy.ndimage`, scikit-image, SymPy, exact/soft accuracy, shape/color/object helper utilities, and safe visualization/text summaries. Object tools are optional; the prompt must not force a particular decomposition method.
- Action protocol: tolerate plain Python fenced blocks or a small `PYTHON:` envelope, execute externally, return concise stdout/error/metric observations, then continue the same Qwen context. This matches our earlier strict-JSON-vs-tolerant-parser ablation rather than assuming guided JSON is always beneficial.
- Child calls become ordinary batched Qwen generations. A parent submits 1-2 compact child packets; vLLM can process those sequences under the same `max_num_seqs` budget. No extra 27B checkpoint is loaded.

6.4 Qwen-local delegation policy - cap width by evidence

State	Extra child budget	Child role	Why

Train-exact, low ambiguity, NVARC/Qwen agreement	0	None	Do not spend agentic compute when the candidate is already strongly supported.
Train-exact but plausible alternative mechanic / test extrapolation ambiguity	1	Fresh generalization critic	Challenge constants, orientation assumptions, object identity and output-size rule; return evidence, not a full transcript.
Near-miss code or two train-consistent hypotheses	1-2	Program repair + alternative-hypothesis specialist	Use cell-diff/soft-error evidence to repair one branch while another tries a different rule family.
NVARC vs Qwen-code disagreement / code vs direct-Qwen disagreement	2 max	Two independent mechanism analysts	Target complementary hypothesis coverage; keep children unaware of each other to reduce anchoring.
Still unresolved after children	Stop / route to other mechanism	Squeeze-Evolve / Imbue / Athanor branch	Do not recursively burn the full 10-agent budget in v0.

6.5 Scoped child context is the context-rot defense we should preserve

- Do not clone the parent transcript into every child. Pass only the puzzle data required for the sub-task plus the current hypothesis/code/evidence that the child is meant to challenge.
- Recommended child input packet: `role_goal`, `train_pairs`, optional `test_input`, `current_hypothesis`, optional `current_code`, `observed_failures`, `specific_question`, `budget`.
- Recommended child return packet: `hypothesis`, `evidence_for`, `evidence_against`, optional `code_patch`, `train_fit`, `predicted_grid`, `uncertainty/caveats`. Parent stores this artifact, not the child's hidden working transcript.
- This fits our ~30K practical context far better than long shared conversation trees and also creates clean provenance for later candidate selection.

6.6 Critical source weakness: train-failing programs can still consume pass@2 slots

Do NOT copy ARCgentica selection literally. `solve.py` executes the returned program on train/test and records soft train scores, but a program that misses training examples is still saved. `score.py` / `submit.py` then take the first two non-empty test outputs by attempt order: there is no exact-train-fit filter, no dedup, and no judge/ranking. The committed final result `e12f9a14/attempt_1.json` contains a train failure (~0.9705 soft score on one train example) while still producing a test candidate.

- Our final-candidate gate: `train_exact_fit == 1.0` on every training pair under a hardened external executor. Anything else cannot occupy attempt_1/attempt_2.
- Near-miss programs remain valuable as repair/mutation organisms for Imbue-style evolution; they are search state, not submission state.
- Canonical-grid dedup before pass@2. Two independent attempts that emit the same grid count as one candidate cluster; keep both provenance traces, but do not waste the second slot.
- Selection uses NVARC + Qwen-direct + Qwen-code + child/refinement provenance. Prefer mechanic-diverse coverage over “first two successful outputs”.

6.7 Prompt details to modify before using with Qwen

- Do not copy the mandatory `scipy.ndimage.label` wording. The reference prompt explicitly tells the model it MUST use object labeling; that can bias Qwen toward object decomposition on tasks that are better solved by pattern, arithmetic, topology or global symmetry. Expose the library, not the obligation.
- Keep the valuable constraints: write a general `transform`, verify every train pair, avoid arbitrary constants, consider orientation/symmetry generalization, debug by execution, and prefer simple rules.
- Run a controlled test-awareness ablation. ARCgentica exposes `challenges` during hypothesis search; Noemon deliberately hides test input until the rule is validated. Compare (A) test-aware program synthesis versus (B) train-only induction -> exact verifier -> target reveal on the same frozen subset.
- Do not over-prescribe a long reasoning checklist if Qwen experiments show it hurts. We already have external evidence that hard ARC tasks can degrade under rigid decomposition. Keep a minimal prompt variant in every ablation.

6.8 Program execution: keep ARCgentica simplicity, add Confluence hardening

- Reference implementation already uses a fresh temporary directory, stdin JSON, a child Python subprocess and a 5-second wall timeout. That is a good base.
- Our runner should additionally create a process group/session and kill the full tree on timeout; apply CPU/memory/file-size limits where available; sanitize stdout; cap output size; reject non-rectangular / non-0..9 grids; never reuse mutable process state between candidate programs.
- Kaggle internet-off already removes network egress, but generated code should still be treated as untrusted local code with least privilege.

6.9 Integration with NVARC and the other portable mechanisms

Existing mechanism	ARCgentica contribution	Combined role
NVARC 4B TTT/DFS specialist	Qwen code-mode hypothesis/program branch	Use disagreement as a trigger for scoped children; measure unique GT union, not only standalone Qwen score.
Confluence	Persistent REPL + dynamic delegation	Confluence supplies external exact verifier/hardened runner; ARCgentica supplies interactive exploration and targeted child contexts.
Noemon	Code-mode branch beside test-blind rule branch	Ablate test-aware vs train-only reasoning; both feed the same exact candidate gate.
Athanor	Focused child can act as artifact critic	Fresh critic receives code/hypothesis/train metrics only; no need to expose the full parent transcript.
Imbue	REPL near-miss programs become mutation seeds	Repair high-soft-score failures rather than discard useful partial programs.
Squeeze-Evolve	Delegation count becomes adaptive compute route	Consensus/easy tasks get no child; high disagreement routes to 1-2 children or a

		stronger XHIGH/code profile.

6.10 AGT-Q27 experiment ladder - isolate REPL and delegation gains

Experiment	Add	Budget discipline	Primary metric
AGT-Q27-G0	Persistent Python REPL + program synthesis + external full-train exact gate; no children	One parent Qwen context; bounded Python turns; no recursive delegation	Standalone; NVARC union oracle; train-exact program rate; near-miss rate
AGT-Q27-G1	One optional fresh child critic when parent is train-exact but ambiguity/disagreement remains	0 child on confident cases; <=1 child otherwise	Unique GT additions; critic rescue vs corruption; marginal tokens/second
AGT-Q27-G2	Up to two parallel scoped children: alternative hypothesis + program repair/generalization	Only hard/disagreement group; no grandchildren	Oracle-union gain per child; output cluster diversity; NVARC-complementary GTs
AGT-Q27-G3	Union of Qwen direct + parent/child code candidates + NVARC; canonical dedup + provenance-aware pass@2	No extra generation beyond G2; selection experiment only	GT-present-but-selected; pass@2; selector recovery/corruption
AGT-Q27-G4	Adaptive depth-2 child recursion only when G2 demonstrates repeatable unique gains	Strict per-task call/token/time cap; stop on exact/provenance-diverse consensus	Solved/task-hour; marginal gain of recursion; 12h feasibility

6.11 Metrics required before keeping ARCgentica mechanics

- Program search: full-train exact rate, best soft score before repair, near-miss -> exact repair conversion rate.
- Delegation: child-call rate, average/max child count, unique hypothesis families, marginal oracle gain per child call.
- Complementarity: NVARC-only solved, Qwen-code-only solved, overlap, union oracle and unique GT additions by EDA tier/failure segment.
- Selection: canonical candidate count, duplicate ratio, provenance-diverse candidate count, GT-present-but-not-selected.
- Prompt policy: train-only vs test-aware delta; minimal vs prescriptive prompt delta; object-helper use versus forced-object heuristic.
- Runtime: Qwen generation tokens, Python executions, timeout/error rate, wall-clock, GPU utilization and solved/task-hour under 4xL4.

6.12 What we deliberately do NOT port

- Agentica Server/cloud provider plumbing, external API routing, cache billing/provider details or the original high-concurrency server configuration.
- Ten logical agents as a fixed quota. Start 0/1/2; recurse only after measured value. The reference itself averaged far below the cap.
- Multiple resident copies of Qwen3.8-27B. All parent/children are logical contexts backed by one TP4 model instance.

- “First two non-empty outputs” submission selection, train-failing programs, or duplicate grids consuming both ARC attempts.
- Mandatory `scipy.ndimage.label` / object-centric decomposition. Libraries are tools, not a required theory of every puzzle.
- Full parent transcript cloning into children. Pass scoped task state and preserve compact child artifacts/provenance instead.

6.13 Source links used for the portable section

Repository snapshot audited:
https://github.com/symbolica-ai/arcgentica/tree/bbba64d047e7e043619e50e336dd75c60f234d00

README / reported result / final-run overview:
https://github.com/symbolica-ai/arcgentica/blob/main/README.md

Agent wrapper / recursive scoped delegation:
https://github.com/symbolica-ai/arcgentica/blob/main/arc_agent/agent.py

Initial + child prompts: https://github.com/symbolica-ai/arcgentica/blob/main/arc_agent/prompts.py

ARC types / exact + soft verifier:
https://github.com/symbolica-ai/arcgentica/blob/main/arc_agent/types.py

Attempt loop / external program execution:
https://github.com/symbolica-ai/arcgentica/blob/main/solve.py

Pass@2 selection behavior: https://github.com/symbolica-ai/arcgentica/blob/main/score.py

Submission builder: https://github.com/symbolica-ai/arcgentica/blob/main/submit.py

Run summary / attempt accounting: https://github.com/symbolica-ai/arcgentica/blob/main/summary.py

Exact final configuration:
https://github.com/symbolica-ai/arcgentica/blob/main/output/2025/anthropic/claude-opus-4-6/final/config.json

Representative parent delegation trace:
https://github.com/symbolica-ai/arcgentica/blob/main/output/2025/anthropic/claude-opus-4-6/final/logs/0934a4d8/0/0/agent-0.log

Representative scoped child trace:
https://github.com/symbolica-ai/arcgentica/blob/main/output/2025/anthropic/claude-opus-4-6/final/logs/0934a4d8/0/0/agent-1.log

Committed train-failing candidate evidence:
https://github.com/symbolica-ai/arcgentica/blob/main/output/2025/anthropic/claude-opus-4-6/final/results/e12f9a14/attempt_1.json

Agentica Python SDK runtime: https://github.com/symbolica-ai/agentica-python-sdk

Official technical article: https://www.symbolica.ai/blog/arcgentica

7. PiVOTools Agentic Coding - stateful Python REPL + interleaved program synthesis

Portable takeaway: PiVOTools is the cleanest way to isolate the value of agentic coding before adding sub-agents, reviewers or evolutionary search. The intervention is simple: let one reasoning model repeatedly inspect a persistent Python workspace, execute diagnostics, observe results/errors, revise its hypothesis, and finally emit one self-contained transformation program. The public-eval results are self-run rather than semi-private verified, so the evidence we use is the same-model delta rather than the headline score.

Implementation-order change: run a minimal PiVOTools-style Qwen3.8 REPL baseline BEFORE ARCgentica delegation. If one Qwen27 + stateful Python already adds unique GT coverage over NVARC/direct-Qwen, only

then pay for scoped child agents.

7.1 Evidence that the REPL regime itself matters

Model / evaluation slice	Plain reasoning	Agentic coding	What we use from it
GPT-OSS-120B High, full public 120	6.11%	26.38%	Large same-model gain from interleaved Python; directly relevant because this was also self-hosted through vLLM.
MiniMax M2.1, full public 120	3.06%	10.56%	Effect appears across a second model family; do not treat it as GPT-specific.
GPT-5.2 XHigh, common 107-task slice	59.81%	73.36%	REPL gain persists even from a strong reasoning baseline.
GPT-5.2, all 120 counting missing/timeouts as failures	53.33%	68.33%	Useful robustness view; still public evaluation, not a verified semi-private score.

Interpretation for our project: do not assume Qwen3.8 needs a complex multi-agent framework to show an agentic gain. First test the smallest causal intervention - persistent executable state plus a tool loop - under the same Qwen checkpoint, task list and scoring harness as direct XHIGH.

7.2 The portable core loop - one Qwen, one persistent workspace

Stage	PiVOTools behavior	4xL4 Qwen3.8 port
1. Initialize	Puzzle is present in the prompt and also preloaded as a Python variable.	Create one persistent worker namespace per active puzzle lineage; preload puzzle and approved libraries.
2. Reason	Model forms a hypothesis and may call Python instead of committing to a final answer.	Qwen Thinking XHIGH decides whether to inspect, measure, prototype or verify.
3. Execute	Python tool runs inside a stateful REPL; imports, variables and helper functions persist.	Run tool snippets in a long-lived local worker; do not reload the 27B model or restart the namespace every call.
4. Observe	stdout, runtime errors and timeout messages are returned to the model.	Feed concise tool result/error back as a tool response; failures become repair signals rather than silent candidate loss.
5. Revise	The same logical conversation continues through multiple code calls.	Preserve Qwen reasoning/tool history until a context governor must compact it.
6. Finalize	Model returns one code block defining solution(grid).	Require one self-contained solution() with imports/helpers included; no hidden dependency on mutable REPL variables.
7. Gate	Final program is independently rerun on all train pairs; any failure rejects it.	Fresh-process full-train exact gate before the candidate is allowed into the NVARC/Qwen pool.

Important distinction: PiVOTools GPT-OSS config sets API stateful=False, but the Python REPL is still stateful. Their client manually carries reasoning/tool state across calls. For us the invariant is persistent reasoning + persistent Python state, not a specific Responses-API flag.

7.3 Why Qwen3.8-27B is unusually compatible with this design

- Our exact chat template already supports tools and multi-step tool history. Official Qwen3.8 formats tool calls inside <tool_call><function=...>...</function></tool_call> and tool outputs inside <tool_response>...</tool_response>.
- Qwen3.8 supports Thinking and reasoning_effort=xhigh, which matches the role PiVOTools gives to a reasoning model between code executions.
- Preserved thinking is enabled by default in Qwen3.8 and is explicitly described as beneficial for agent scenarios. This aligns with PiVOTools' warning that dropping prior reasoning across tool calls causes soft failures.
- We do not need an OpenAI Responses server. The current direct vLLM LLM(...) runtime can render the Qwen tool schema, parse the XML tool call, execute Python locally, append a tool-response message, and generate again.
- The one constraint is our practical ~30K context. Therefore preserved-thinking starts ON, but a context governor must compact old raw tool outputs before they crowd out verified state.

7.4 Local Kaggle architecture - do not copy Docker/IPyBox

Qwen3.8 XHIGH -> tool-call parser -> persistent Python worker -> stdout/error/timeout -> Qwen3.8 XHIGH -> ... -> self-contained solution() -> fresh exact verifier -> test grids -> candidate pool

Component	Local implementation rule	Reason
Persistent worker	One long-lived subprocess or multiprocessing worker per active lineage with its own namespace.	Retains helper functions/measurements without token re-description.
Preloaded state	puzzle + minimal libraries; optionally ARC helper functions.	Matches the useful part of the REPL without framework dependency.
Timeout	Hard per-tool execution timeout; start much lower than PiVOTools' 120 s and only relax for justified operations.	Kaggle has a fixed 12-hour budget; runaway code must not consume it.
Crash recovery	Kill/restart a poisoned worker; optionally replay a small checkpoint of verified helper definitions.	Python generated by an LLM can hang or corrupt state.
Protected puzzle	Reject attempts to overwrite the puzzle object and suppress whole-puzzle printing.	Preserves ground truth inputs and prevents large context-wasting dumps.
Fresh final verifier	Run final solution code outside the exploratory REPL.	Catches hidden state dependencies and makes the artifact reproducible.

7.5 Exact final-program gate - stronger than “train score looks good”

A Qwen code candidate enters the union only after an independent verifier checks all training pairs. The gate should be stricter than the minimum public implementation:

- Call solution(copy.deepcopy(input_grid)) so the program cannot solve by mutating the supplied grid object.
- Require a rectangular 2D list/array, integral cells, valid ARC colors 0-9 and a bounded legal grid shape.
- Require exact shape and exact cell equality for EVERY training pair. Soft cell accuracy is diagnostic only and never an acceptance criterion.
- Run with a hard timeout and capture exceptions. A runtime error, malformed grid or timeout is a rejected final program.
- After the train gate, execute the same self-contained program on every test input. Multi-query tasks therefore share one rule implementation rather than independent prose guesses.

7.6 Tool errors should become evidence, not failed runs

The tool loop deliberately returns syntax/runtime/timeout feedback to the model. This is highly portable and valuable for Qwen: a wrong array index, missing import or failed assertion should be a short tool response that invites repair. Malformed tool-call arguments should likewise get one concise correction message. Infrastructure/model failures are different: retry them without pretending they are evidence about the puzzle.

Failure type	Return to Qwen?	Local handling
Python syntax/runtime error	Yes	Short traceback tail + error class; let Qwen repair.
Assertion/train mismatch during exploration	Yes	Return mismatch index and compact shape/cell-diff summary.
Tool timeout	Yes	Terminate code; state may need restart; tell Qwen which call timed out.
Malformed tool-call schema	Yes, once	Parser emits expected XML/schema; retry generation.
vLLM/worker/NCCL infrastructure error	No puzzle feedback	Retry/recover at runtime layer; do not contaminate reasoning.

7.7 Prompt/interface details worth preserving - and one thing to ablate

- Preload puzzle in Python so the model can inspect dimensions, colors, components and invariants without rewriting arrays into code.
- Explicitly discourage printing the entire puzzle. Ask for targeted slices, counts, object summaries and helper functions instead.
- Encourage reusable functions because the REPL is persistent; this converts exploratory work into progressively better tooling for the same puzzle.
- Final response should be only one self-contained solution() program. Interleaved exploratory code is not the submission artifact.
- PiVOTools shows all test inputs while reasoning. Noemon suggests train-first/test-blind rule learning may sometimes generalize better. Treat this as a controlled ablation, especially on our multi-query EDA segment.
- Do NOT assume the long PiVOTools ARC-prior prompt is optimal for Qwen. Compare a detailed-prior prompt against a minimal tool-oriented prompt under identical compute.

7.8 Preserved-thinking reliability is a first-class experiment

PiVOTools explicitly identifies inference-stack “soft failures” when reasoning state is lost between tool calls. Qwen3.8 gives us a model-native way to test this. For each lineage, log whether later reasoning actually refers to earlier hypotheses/tool evidence instead of resetting after each Python result.

Mode	History policy	What to measure
PRESERVE	Qwen preserve_thinking=True; retain recent reasoning and tool outputs.	Tool-call depth, hypothesis continuity, solve rate, max context.
COMPACT	After verified milestones, compress old trace into invariants + helper code + failure signatures.	Whether equal/better coverage is retained with less context.
Failure signature	Model repeatedly re-derives the same facts, forgets successful helper functions or	Count these as agent-loop failures, not “model cannot solve ARC.”

	terminates right after first tool call.	

7.9 Pass@2 adaptation for our NVARC system

PiVOTools public evaluation normally uses two independent samples and ranks exact grids by frequency when combining more samples. We should not copy that budget allocation or final selector directly.

- First port: ONE Qwen agentic-code lineage per selected task, because NVARC already supplies an independent specialist distribution.
- Second Qwen code lineage is adaptive only: unresolved NVARC/Qwen disagreement, near-miss, low candidate diversity, hard EDA tier or first code lineage rejected by the train gate.
- Exact-grid deduplicate across NVARC, Qwen-direct and Qwen-program candidates but preserve source/program/trace provenance.
- Do not use raw frequency as the final pass@2 selector. Later use the mechanic-diverse selector/reviewer strategy already recorded in this note.

7.10 PIV-Q27 experiment ladder - isolate the causal gain

Experiment	What changes	Bounded starting budget	Keep only if
PIV-Q27-P0	One Qwen XHIGH lineage + persistent Python + tool loop + self-contained final solution + fresh all-train exact gate.	Start with ~6-8 tool calls; one lineage; same frozen proxy subset as direct baseline.	Adds standalone solves or unique NVARC-union GTs at acceptable task-hour cost.
PIV-Q27-P1	More tool depth only for P0 unresolved tasks.	Raise to ~12 calls on the hard subset, not globally.	Marginal unique-GT gain per extra call is positive.
PIV-Q27-P2	Prompt ablation: detailed ARC priors vs minimal tool-oriented prompt.	Identical tasks/call cap/sampling.	One prompt wins reproducibly or different EDA tiers justify routing.
PIV-Q27-P3	Preserved-thinking vs compact verified-state governor.	Same candidate budget.	Compact mode saves context without losing oracle-union coverage, or preserve mode clearly solves more.
PIV-Q27-P4	Adaptive second independent Qwen code lineage.	Only disagreement/near-miss/hard cases.	Provides non-duplicate correct candidates often enough to justify compute.
PIV-Q27-P5	Union NVARC + Qwen direct + verified Qwen programs; no new generation method.	Selection study only.	Pass@2 improves beyond each branch and oracle gap remains actionable.

7.11 Metrics required before we keep the mechanism

Metric family	Required measurement
Correctness	Standalone exact accuracy; train-perfect program rate; test accuracy conditioned on train-perfect.
Complementarity	NVARC+PIV oracle union; Qwen-direct+PIV union; unique GT additions; GT-present-but-not-selected.
Agent loop	Tool calls/task; useful vs redundant calls; parser failures; Python errors/timeouts; final-code parse/rejection rate.
Generalization	Train-perfect-but-test-wrong rate; multi-query cross-test consistency; rule overfit signatures.

Compute	Max context; input/output tokens; wall-clock/task; solved/task-hour on 4xL4.
EDA slices	Easy/static, Tier-2/3, dynamic, high-ratio and multi-query deltas.
Diversity	Exact candidate duplicates and provenance-mechanic diversity across NVARC/direct/code lineages.

7.12 What we deliberately do NOT port

- GPT-OSS-specific Harmony/vLLM patches. Qwen has its own tool-capable chat template; model-specific patches are unnecessary risk.
- OpenAI Responses API, provider abstraction or Daytona cloud infrastructure. Our Kaggle target is offline direct vLLM.
- Docker/IPyBox as a requirement. Preserve the stateful-REPL semantics with a lightweight local worker compatible with Kaggle.
- A fixed 120-second Python timeout. Start with tighter task-budget-aware limits and relax only when a puzzle operation requires it.
- The entire dependency stack by default. Start with numpy/scipy/scikit-image/networkx/Pillow; expose Z3/OR-Tools only if an experiment proves value.
- Always two expensive Qwen code samples per task. The second lineage is adaptive because NVARC already supplies another generator.
- Frequency-only majority voting as final selection.
- Any final program that only works because of hidden REPL state. Fresh-process verification makes self-containment mandatory.

7.13 Source links used for the portable section

Technical article / same-model ablations:
https://pivotools.github.io/pivotools-quarto-blog/posts/agentic_coding_arc_agi/

Repository snapshot audited:
https://github.com/gutfeeling/arc-agi-2-submission/tree/b3d4494892c6d85fa2a35d548271fb02ea3e191b

Raw public-eval experiment traces: https://huggingface.co/datasets/arcagi2/arcagi2-agentic-coding-publication

Agentic solver + post-hoc exact train gate:
https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/solver.py

Turn parser / final solution extraction:
https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/turn.py

Agentic coding prompt: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/prompts/agentic_coding_solver.txt

Python tool system prompt: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/prompts/system_prompt.txt

REPL tool + protected puzzle/error feedback:
https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/tools/repl_tool.py

Stateful REPL sandbox implementation:
https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/sandbox/ipybox_sandbox.py

REPL/Sandbox state semantics: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/sandbox/base.py

Independent train verifier/test executor:
https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/utils/solving_utils.py

Exact train verification code: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/utils/code_templates/check_solution_on_training_examples.py

Puzzle serialization / all-test mode: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/utils/puzzle_utils.py

Independent samples + frequency pass@2 combiner: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/evaluation/evaluate.py

GPT-OSS self-hosted vLLM config: https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/config/gpt_oss_120b_high.py

Qwen3.8 FP8 official model card / preserved thinking: https://huggingface.co/Qwen/Qwen3.8-27B-FP8

Qwen3.8 official tool-call chat template: https://huggingface.co/Qwen/Qwen3.8-27B-FP8/blob/main/chat_template.jinja

Portable-only rule remains frozen: future additions must directly affect our 4xL4 offline Qwen3.8-27B + NVARC implementation or experiment design; provider/cloud/cost trivia stays out.

8. Poetiq - stateless iterative code refinement with deterministic feedback

Verified reference: Poetiq's released ARC harness was independently evaluated on ARC-AGI-2 Semi-Private at 54.0%. For our project, the useful part is the open-source harness itself: repeated code generation, deterministic execution feedback, compact memory of strong partial programs, and diversity-aware multi-expert aggregation. The private Poetiq meta-system that optimized the harness is not public, so this section ports only mechanics visible in the released code.

8.1 Portable core: fresh Qwen calls, compact evolutionary memory

- Each refinement iteration is a fresh solver call rather than one ever-growing chat transcript. This is highly compatible with our practical ~30K Qwen context ceiling.
- The current puzzle is re-serialized every iteration; selected earlier programs are appended as compact artifacts: code + deterministic evaluation feedback + numeric train score.
- The released default allows up to 10 iterations and retains up to 5 prior partial programs, but our first 4xL4 experiment should start much smaller and expand only if the improvement curve justifies it.
- If a generated transform() becomes exact on every training example, the lineage stops immediately and the verified program is used on the test inputs.

8.2 Deterministic feedback is the main refinement signal

- Correct training examples are marked explicitly as solved; the model does not need to infer whether a program improved.
- Wrong output shape is reported as a shape mismatch rather than a vague failure.
- When shape matches, feedback exposes a cell-level diff: correct cells stay unchanged, while wrong cells are encoded as prediction/correct, plus a soft cell-accuracy score.
- Runtime/parse errors are returned verbatim as repair evidence. This turns failed code executions into useful search information instead of wasting the iteration.

For Qwen, this feedback should be produced outside the model by our verifier. The model should never score its own train correctness.

8.3 Keep only the best partial programs - not the whole history

- Poetiq ranks partial programs by mean train soft score and injects only a bounded top-K set into the next prompt.
- With improving_order=True, selected examples are shown from weaker to stronger, giving the next call a compact trajectory of what was fixed rather than a chronological transcript.

- For Qwen we should preserve program provenance and exact verifier output, but aggressively drop repetitive prose. This is a second context-management strategy to compare against PiVOTools' persistent REPL history.

8.4 Cheap diversity: shuffled demonstrations + independent seed lineages

- Training-example order is reshuffled across refinement iterations. This is a low-cost way to perturb the model's hypothesis path without changing the task.
- Multiple experts use disjoint seed ranges and are solved independently. On 4xL4 we should not run many full Qwen lineages globally; a second lineage should be conditional on unresolved / near-miss / hard tasks.
- Distinct test-output grids are grouped before ranking so duplicate lineages do not consume both ARC submission slots. Preserve this diversity-first ordering principle.

8.5 Qwen-local adaptation: the clean ablation against PiVOTools

PiVOTools and Poetiq give us two different ways to spend Qwen test-time compute on code:

- PIV-Q27: stateful interleaved reasoning -> Python tool -> observation -> continued reasoning in one persistent workspace.
- PTQ-Q27: stateless fresh Qwen call -> final program -> external verifier -> compact best-partial feedback -> fresh Qwen call.

Run them on the exact same frozen task subset and token/time budget. If PTQ matches PIV union coverage with lower context growth, it is the safer Kaggle-wide default; if PIV uniquely solves tool-inspection tasks, route only those cases to the persistent REPL path.

8.6 PTQ-Q27 experiment ladder

- PTQ-Q27-T0: one XHIGH Qwen lineage, 3-4 refinement rounds maximum, K=1 prior partial program, exact train verifier, hard subprocess timeout.
- PTQ-Q27-T1: K=3 best partial programs in improving order. Keep only if unique NVARC-union coverage rises without context blow-up.
- PTQ-Q27-T2: reshuffle train example order per iteration and launch a second seed lineage only for unresolved / near-miss / EDA-hard tasks.
- PTQ-Q27-T3: pool verified Poetiq programs with NVARC + Qwen-direct + PIV candidates; do not use Poetiq's raw first-two output policy as our final selector.

8.7 Metrics required before keeping Poetiq mechanics

- Train-perfect program rate and iteration-to-first-pass.
- Best train soft-score curve by iteration, including how often improvement stalls or regresses.
- Standalone exact accuracy, NVARC+PTQ oracle union, and unique GT additions over NVARC/PIV/direct-Qwen.
- Prompt/context tokens per round, wall-clock per task, code-execution failures/timeouts, and EDA-tier breakdown.
- Pass@2 diversity: how often the second distinct program/output adds a correct answer rather than duplicating the first mechanic.

8.8 Port boundary that matters

The released repository exposes the ARC iterative-coding harness, not Poetiq's private meta-system that automatically designed/optimized that harness. Our experiment therefore tests the released mechanism only. Also, do not initially merge votes from train-failing programs into a

train-passing output group; that source option may inflate confidence without adding independent verified evidence.

8.9 Source links used for the portable section

Official verified result: https://poetiq.ai/posts/arcagi_verified/

Official approach overview: https://poetiq.ai/posts/arcagi_announcement/

Open-source solver: https://github.com/poetiq-ai/poetiq-arc-agi-solver

Iteration + feedback loop:
https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/solve_coding.py

Multi-expert diversity-first grouping:
https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/solve_parallel_coding.py

Released configuration:
https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/config.py

Feedback prompt: https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/prompts.py

Local subprocess executor:
https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/sandbox.py

9. Johan Land - modality-driven search + holistic trace judging

Verified reference: 72.9% ARC-AGI-2 Semi-Private and 76.1% Public in the paper. The central portable lesson is not the proprietary frontier-model mixture; it is the separation of independent candidate-generation families from a fresh-context holistic selector that sees candidate traces together and can recover a plausible minority mechanic.

9.1 Why this matters for our current bottleneck

- The paper reports an 86.23% candidate-pool oracle on 167 public test instances, but only 76.65% instance-level pass@2 after selection: candidate generation and candidate selection are distinct failure modes.
- Reported error accounting separates 22 generation failures (no correct candidate existed) from 17 selection failures (a correct candidate existed but was not selected). This is exactly the distinction we already need for NVARC+Qwen union analysis.
- The paper’s same-candidate-pool post-hoc selector comparison reports seven additional solved public instances over majority voting, all from minority hypotheses. This was not a separately rerun end-to-end generation experiment; synthesis added one further solved instance where no original candidate exactly matched.

The paper's public snapshot used 167 instances, while our current Kaggle evaluation harness exposes a different count. Use these numbers as mechanism evidence, not as a directly comparable score denominator.

9.2 Replace “modalities” with our independent generator families

Our current Qwen notebook uses language_model_only=True, so we should not force an image branch. The portable abstraction is independent search families with different failure biases:

- Family A - NVARC 4B TTT/DFS specialist: strong exact-grid prior and augmentation-based scoring.

- Family B - Qwen3.8-27B direct OFF/XHIGH: broad rule induction and alternative hypotheses.
- Family C - Qwen executable-code solver: PiVOTools stateful REPL and/or Poetiq stateless refinement, with exact train verification.
- Optional later family - Athanor/Noemon/Imbue repair or reviewer outputs only when the task remains ambiguous.

The key is that these families generate independently. Do not feed one shared hint or majority hypothesis into every branch before generation, because that destroys the complementarity we are trying to buy.

9.3 Exact-grid dedup, but never discard trace/provenance

- Johan's state groups identical predicted grids, counts how many runs produced them, and separately stores the full reasoning trace for every run.
- Our candidate record should likewise canonicalize the grid once while retaining every source: NVARC scorer/augmentation provenance, Qwen direct rule trace, code program, train-verification status, repair history and generation profile.
- This directly fixes the common mistake “duplicate grid = duplicate evidence.” Two independent mechanisms producing the same grid are stronger evidence than repeated sampling from one collapsed lineage.

9.4 Holistic judge: one fresh Qwen context that compares mechanics jointly

- The final V7 selector builds one prompt containing the ARC examples, test input, each candidate grid and the reasoning/program trace associated with that candidate.
- The judge is asked to identify two solutions representing the most plausible mechanics, not simply score each grid independently. This lets evidence from one candidate expose flaws or useful sub-rules in another.
- For our first local port, use one fresh Qwen XHIGH judge sequentially after generation. Do not load another 27B instance; reuse the resident TP4 checkpoint.
- Because our practical context is ~30K, pass compact artifacts rather than raw multi-thousand-token histories: concise rule, final grid, verifier result, key failure/repair evidence, and provenance family.

9.5 Minority protection and judge synthesis

- Attempt 2 should intentionally cover a different plausible mechanic when evidence remains ambiguous. Do not make it “frequency rank-2.”
- Permit the judge to propose a new grid only in a controlled ablation. Tag it judge_synth, validate shape/color/domain constraints, preserve the synthesis rationale, and measure wrong-synthesis rate separately.
- Never let synthesis erase the best existing verified candidate. ARC gives two slots, so synthesis should compete for one slot under explicit evidence, not replace both.

9.6 Early stopping must be provenance-aware, not raw consensus

V7 has a raw exact-grid consensus stop (top group >25%, count >=11, every other group count=1). We should not copy that threshold because our NVARC experiments already show duplicate/search collapse. A safe local stop should require agreement across genuinely independent provenance families, no strong alternate mechanic, and no hard/near-miss routing flag. Otherwise continue to code or reviewer branches.

9.7 Prompt discipline: preserve diversity

- The paper's negative results show prescriptive step-by-step templates, shared hints, forced decomposition and iterative refinement can reduce hypothesis diversity. Keep generation branches independent and minimally anchored.
- CSV grid serialization performed well in the reported prompt ablations. Treat CSV vs our current format as a controlled Qwen serialization experiment, not an assumed universal win.
- The holistic judge is the right place to combine traces. Do not combine them before generation.

9.8 JUD-Q27 experiment ladder

- JUD-Q27-J0: candidate pool = NVARC + Qwen direct + verified Qwen programs; exact-grid dedup; select at most 6-8 mechanic/provenance-diverse grids; one fresh Qwen XHIGH judge chooses two distinct outputs. Judge cannot synthesize yet.
- JUD-Q27-J1: enable one judge-synthesized grid as an eligible second-slot candidate; track synth recoveries and synth errors separately.
- JUD-Q27-J2: only on ambiguous tasks where J0 helped, run three independent judge calls and aggregate first-choice=2 / second-choice=1 as in V7. Do not spend this globally. The paper uses three judges, but does not isolate the marginal causal gain of three judges versus one, so J0 remains the correct first ablation.
- JUD-Q27-J3: provenance-diverse early-stop gate. Skip judge only when at least two independent generator families agree and no strong alternate/hard flag remains.

9.9 Metrics required before keeping holistic judging

- Candidate oracle accuracy before judging and final pass@2 after judging; selection gap = oracle - selected.
- Minority-recovery count: correct answer existed but was not frequency rank-1, and judge recovered it.
- Delta vs current NVARC selector and simple exact-grid majority on the identical candidate pool.
- Judge-synth recovery count and wrong-synthesis rate.
- Judge tokens/context/wall-clock, plus breakdown by EDA tier, NVARC near-miss status and generator provenance.

9.10 Source links used for the portable section

Paper: https://arxiv.org/abs/2606.31543

Released V7 solver branch: https://github.com/beetree/ARC-AGI/tree/Johan_Land_Solver_V7

Staged independent generation:
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/solver/steps.py

Candidate dedup + provenance/trace store:
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/solver/state.py

Holistic selector + synthesis:
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/selection_advanced.py

Holistic duo-pick prompt:
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/audit_prompts.py

Judge parser/execution:
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/judges.py

Raw-consensus early-stop source (adapt, do not copy thresholds):
https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/selection.py

10. CoreThink - compositional neuro-symbolic reasoning

Reference result: ~24.4% ARC-AGI-2 Public Eval; reported ensemble ~30.8%. This section keeps only mechanics that can be ported to our 4xL4 offline Qwen3.8-27B + NVARC system.

> **Port thesis: Do not copy the whole CoreThink stack. Build a small symbolic specialist branch: deterministic object features -> bounded mechanic hypotheses -> cross-example consistency -> Qwen reasoning/code. This should remain an independent hypothesis family from NVARC/Qwen-direct.**

10.1 Portable mechanics to keep

- Deterministic object layer (HIGH): Compute connected components, bounding boxes, centroids, colors, holes, symmetry, and relative positions locally. Do not make the LLM perform raw perception arithmetic.
- Bounded unit-pattern vocabulary (HIGH): Use a compact, inspectable mechanic set covering move, reflect, rotate, crop, recolor, repeat, connect, fill, inside/outside, and relative placement.
- Repeated proposal + vote (MEDIUM): Produce a few stochastic mechanic proposals from the same object transition and aggregate agreement over mechanic names/parameters.
- Cross-example intersection (HIGH): Score the consistency of mechanics that survive across every training pair, eliminating one-example coincidences.
- Symbolic hint one branch only (HIGH): Give the top 1-3 hypotheses to one separate Qwen branch; do not anchor every generator with the same shared hint.
- Executable confirmation (HIGH): If a symbolic hypothesis becomes code, require the all-train exact verifier; a symbolic score is not final truth.

10.2 Source-code findings that change our port

- The pattern detector uses PATTERN_DETECTION_REPETITIONS=5 by default and aggregates repeated detections with a Counter. For the Qwen-local version, do not start with five full XHIGH calls; begin with deterministic feature extraction plus 2-3 bounded proposals.
- `patterns_intersection/aggregate.py` aggregates pattern names/parameters/reasons across results with Counters. Our score should be cross-example mechanic consistency, not model confidence.
- The object representation explicitly carries bounding boxes, centroids, colors, and masked-grid structure. Compute these deterministically with NumPy/SciPy/skimage.
- The paper's ensemble improvement is evidence of complementary value. Crucially, the reported meta-classifier does not synthesize new grids: it selects two outputs without replacement from a four-candidate pool (2 CoreThink + 2 ARC Lang). Therefore the result directly supports coverage-first, judge-later ordering. Evaluate CORE-Q27-C0 primarily by unique oracle-union additions over NVARC+Qwen rather than by standalone score.

10.3 CORE-Q27-C0 - minimal experiment

1. On the frozen hard/proxy subset, run a deterministic component/object extractor and build a compact feature summary.

2. For each training pair, generate bounded mechanic candidates; deterministic candidates first, Qwen proposals second.

3. Use a cross-example consistency filter to retain the top 1-3 mechanics.

4. Give a fresh Qwen XHIGH branch the training examples plus the symbolic hypothesis pack and let it produce the final rule/code candidate.

5. If the candidate is code, apply an all-train exact gate in an isolated subprocess.

6. Even when exact-grid deduplication merges outputs, preserve symbolic provenance/trace and keep branch identity distinct from NVARC/Direct/PIV/PTQ.

10.4 Do not port

- Do not begin with a large hand-written DSL or hundreds of hard-coded solvers.
- Do not copy a cell-wise majority/consensus grid as the final answer.
- Do not inject the same symbolic hint into every Qwen branch - this risks diversity collapse.

10.5 Keep/drop metrics

- Unique exact solves + NVARC/Qwen union oracle delta ǀ
- Symbolic hypothesis precision and false-hint rate ǀ
- Near-miss (>90% cell accuracy) -> exact recovery rate ǀ
- Latency/tokens per rescued task; hard-tier/dynamic/high-ratio breakdown ǀ

10.6 Source links

Paper: https://arxiv.org/abs/2604.02434

GitHub: https://github.com/CoreThink-AI/arc-agi-2-reasoner

Repeated pattern proposals:
https://github.com/CoreThink-AI/arc-agi-2-reasoner/blob/main/arc_agi/src/patterns/find_patterns.py

Cross-example pattern aggregation:
https://github.com/CoreThink-AI/arc-agi-2-reasoner/blob/main/arc_agi/src/patterns_intersection/aggregate.py

Object representation:
https://github.com/CoreThink-AI/arc-agi-2-reasoner/blob/main/arc_agi/src/objects/base.py

11. Jeremy Berman - evolutionary test-time compute with natural-language instructions

Reference result: 29.4% ARC-AGI-2 Semi-Private, verified by ARC Prize. Portable focus: evolve natural-language rules, score them by fresh follower execution, then revise/synthesize the best rules.

> **Port thesis: LANG-Q27 will be a code-free mechanic-search branch. Qwen writes a rule and a fresh-context Qwen follower executes it. This measures hypothesis quality without syntax/program bugs and can produce a failure mode different from NVARC/PIV/PTQ.**

11.1 Exact portable loop from source

- Instruction generation: Generate several natural-language transformation rules from the training I/O plus test inputs. Local start = 4 candidates, not dozens.
- Leave-one-out follower scoring: Treat each training example as a pseudo-test; give the follower the other training examples plus the candidate instruction and ask it to solve the held-out input. The target output is not included in the follower prompt.
- Deterministic score: exact match primary; same-shape cell similarity soft signal; shape mismatch 0 ł
- Individual revision: Revise a rule using its wrong outputs/diffs. Local start = top-2 only, one revision each.
- Pooled synthesis: Create one new combined instruction from multiple partial rules plus per-example scores.
- Final diverse replay: Apply top rules to the test input with fresh followers multiple times; retain up to two distinct grids/provenances.

11.2 Important nuance about leave-one-out

- The source implementation may generate an instruction after seeing the full training challenge; during scoring, the held-out example becomes a pseudo-test and is removed from the follower context. This creates strong rule-following/generalization pressure, but instruction discovery itself is not strict N-1 holdout.
- Keep L0 source-faithful. If it gains, run an L1 stricter ablation: generate/validate the instruction from N-1 pairs and test it on the held-out pair - only for ambiguous/hard tasks because compute rises quickly.

11.3 LANG-Q27-L0 - bounded local experiment

1. 4 independent natural-language rules generate; Python/tool disabled ł

2. Compute leave-one-out follower scores for each rule in fresh contexts; start the follower with OFF/short-think.

3. Exact mean + cell-soft mean + shape failures record; raw self-confidence ignore ł

4. Top-2 rules individually revise + one pooled synthesis candidate ł

5. Apply the top 2-3 instructions to the actual test input with fresh followers; preserve the first two distinct grids plus rule provenance.

6. Drop the branch if it adds no unique exact candidate to the NVARC/Direct/PIV/PTQ union.

11.4 Qwen-specific safeguards

- Because the generator and follower are the same Qwen3.8 model, correlated errors are possible. Decorrelate with fresh context, shuffled training order, different reasoning effort/temperature, and optionally alternate serialization.
- [LOCAL DEVIATION FROM SOURCE] ARC Lang's source follower is explicitly told that the instructions are only a guide and that it may ignore erroneous instruction parts to recover the correct grid. Therefore its instruction score partly measures follower reasoning/override ability, not instruction quality alone. Our LANG-Q27-L0 scorer should use a bounded follower that applies the supplied rule and may repair only local execution ambiguity - no unconstrained fresh hypothesis search.
- Do not use hundreds of prompts; the first experiment should be 4 initial rules + 2 revisions + 1 synthesis + bounded final follows.

11.5 Keep/drop metrics

- Instruction score vs actual test correctness calibration ł
- Unique union solves vs Direct/PIV/PTQ/NVARC ł
- Rule diversity: do the top instructions represent distinct mechanics or merely paraphrases?
- Follower instability: output variance when the same rule is followed repeatedly.
- Tokens/wall-clock per exact rescued task ł

11.6 Source links

ARC Prize verified community leaderboard: https://arcprize.org/leaderboard/community

GitHub - ARC Lang: https://github.com/jerber/arc-lang-public

Pipeline README: https://github.com/jerber/arc-lang-public/blob/main/README.md

Scoring / revision / final diversity:
https://github.com/jerber/arc-lang-public/blob/main/src/run.py

Follower execution: https://github.com/jerber/arc-lang-public/blob/main/src/main.py

12. Eric Pang - efficient evolutionary program synthesis with a growing library

Reference result: 26.0% ARC-AGI-2 Semi-Private, verified by ARC Prize. Portable focus: reuse programs learned from public training tasks as task-conditioned priors for new ARC tasks.

> **Port thesis: Do not build the full DreamCoder/LPN stack. Use a frozen public-data program memory: execute library programs on the current task's training pairs, score them with exact + cell-soft metrics, retrieve top diverse programs, and provide them as priors in the Qwen code prompt. This can amortize repeated reasoning compute.**

12.1 Source mechanics worth porting

- [CORRECTED] Growing Python library: Eric Pang's write-up says the best program among the LLM-generated programs is added according to primary exact-example count plus secondary cell-level score; it is not required to be solved, train-perfect, or even "near-solved". Our version: build only from public-training data, re-verify every imported program, then freeze the library during competition inference.
- Task-conditioned retrieval: Execute library programs on current training inputs; primary score = number/fraction of exact training examples, secondary score = mean cell-level accuracy.
- Exploration-aware selection: The source experiments test score sums -> softmax -> stochastic sampling to avoid getting stuck at a top-1 local optimum. Local baseline = top-1; if diversity helps, test top-2/3 sampled retrieval.
- Prompt reuse: Provide retrieved program(s) as priors/examples in a new synthesis prompt, explicitly labeled as not guaranteed correct.
- Failure-diff feedback: Use differences between expected and produced training outputs as a refinement signal in the prompt.
- Library growth: Append the best newly generated program. In our first competition version, allow growth only during the public-data build phase; disable persistent cross-task writes on eval/private tasks.

12.2 Important source findings

- Relevant source code loads saved_library_1000.pkl described as primitives obtained from the 1,000 ARC-AGI-2 public training challenges |
- get_best_primitives executes each primitive on current training inputs and sorts by exact-example count then cell-level score. This is ARC-native retrieval and needs no embedding model |
- Writeup reports final use of score-weighted selection and two selected library programs in the prompt. That is the mechanism worth testing; whole-library prompt dumping is not |
- Library entries are whole Python programs, not necessarily clean abstract reusable subroutines. Our memory therefore needs metadata/dedup/compactness control to avoid irrelevant code bloat |

12.3 LIB-Q27-E0 - minimal frozen-library experiment

1. Build a compact seed library of 100-500 verified transform programs from public training data; do not begin the first test with expensive regeneration over all 1,000 tasks.

2. Run candidate programs on new-task training examples in isolated subprocesses with timeouts; collect exact-count, cell-soft, shape-validity, and runtime metrics.

3. Behavior/code duplicates collapse; top 2-3 diverse programs retrieve ı

4. Provide retrieved programs as priors to the PIV-Q27 or PTQ-Q27 prompt; do not force the model to copy them.

5. No-library vs top-1 vs top-3-diverse controlled ablation ı

6. Keep the inference-time library frozen; disable cross-task persistent growth from eval/private tasks in the initial version.

12.4 Do not port

- Do not build the LPN/JAX latent retriever yet; heuristic execution scoring is cheaper and already ARC-native.
- Do not dump the whole library into the prompt.
- Do not blindly add low-quality/current-task programs to the library - library bloat will cause negative transfer.
- Do not use persistent learning across private/evaluation tasks in the initial Kaggle system; frozen public-data memory keeps evaluation clean and reproducible.

12.5 Keep/drop metrics

- No-library vs retrieval: Qwen exact accuracy + NVARC union-oracle delta ı
- Retrieved program relevance: top-k train exact/soft scores ı
- Token/time savings: whether a retrieved prior reduces Qwen iterations/tool calls.
- Retrieval diversity and duplicate behavior ı
- Negative transfer: failure rate on previously solved tasks caused by introducing a prior.

12.6 Source links

ARC Prize verified community leaderboard: https://arcprize.org/leaderboard/community

GitHub: https://github.com/epang080516/arc_agi

Technical writeup: https://ctpang.substack.com/p/arc-agi-2-sota-efficient-evolutionary

Library/retrieval scoring: https://github.com/epang080516/arc_agi/blob/main/src/logic.py

Submission / saved public-training library:
https://github.com/epang080516/arc_agi/blob/main/src/submission.py

Program data model: https://github.com/epang080516/arc_agi/blob/main/src/models.py

Final implementation order after 12 solver audits

> **Research phase complete. From this point, return to controlled experiments instead of searching for new mechanisms. Rule: do not merge every architecture at once; promote a mechanism only after it demonstrates unique NVARC+Qwen coverage or efficiency.**

1. Complete Qwen3.8-27B TP4 model-load + warmup proof; freeze the direct OFF/XHIGH baseline, parser, and candidate schema.

2. Primary agentic comparison: PIV-Q27-P0 (stateful REPL) vs PTQ-Q27-T0 (fresh-call verifier feedback) on the same frozen hard/proxy subset and the same token/time budget.

3. Direct vs PIV vs PTQ: standalone exact, all-train-pass, NVARC union oracle, unique additions, near-miss recovery, tokens, wall-clock, EDA-tier breakdown compare ׀

4. If the code branch demonstrates gain, run LANG-Q27-L0 as a code-free, complementary natural-language rule-evolution hypothesis family.

5. On NVARC duplicate-collapse/near-miss/object-heavy tasks, run the CORE-Q27-C0 symbolic specialist and LIB-Q27-E0 frozen public-program retrieval as a compute-amortization test.

6. Only mechanisms with unique union gain keep ׀ Similar-output branches merge/disable; provenance/trace always preserve ׀

7. If candidate coverage improves materially, run the JUD-Q27-J0 holistic trace selector; do not add a judge early to mask weak generation.

8. After J0 shows gain, ablate Athanor review/EXPAND, Johan synthesis, Squeeze adaptive compute, Imbue evolution, and ARCgentica delegation only on ambiguous cases.

9. The final local stage-wave scheduler should route easy/consensus tasks through a cheap path and hard/disagreement tasks through extra compute. Keep one resident TP4 Qwen and run logical roles sequentially/batched.

10. End-to-end acceptance: reproducible leaderboard gain above the 32.64 NVARC baseline plus improvement on the proxy/test-like segment without public-eval overfit.

Reserved experiment IDs

- PIV-Q27-P0 - stateful Python REPL - FIRST.
- PTQ-Q27-T0 - fresh-call code refinement - FIRST, paired with PIV.
- LANG-Q27-L0 - natural-language rule evolution - after basic code baseline.
- CORE-Q27-C0 - object/pattern consistency specialist - conditional hard/near-miss branch.
- LIB-Q27-E0 - frozen public-program retrieval - conditional compute-amortization branch.
- JUD-Q27-J0 - holistic trace selection - only after candidate coverage rises.

Source-integrity note

Benchmark/source integrity remains explicit. Jeremy Berman 29.4% and Eric Pang 26.0% are ARC-AGI-2 Semi-Private entries on the ARC Prize Community Leaderboard; ARC Prize states Semi-Private ARC-AGI-1/2 submissions there are run and verified by ARC Prize. CoreThink 24.4%/30.8%, Athanor 95.7%, Squeeze-Evolve 97.5%, Confluence 97.92%, ARCgentica 85.28% and similar public results are public/self-reported unless independently noted. For our 2026 Kaggle experiments, the official metric is output-weighted pass@2: every task test output gets exactly two predictions, either exact match scores 1, and the final score averages over all task test

outputs. Historical public papers/repos sometimes report task-normalized or task-level numbers, so those denominators must not be silently mixed with the current Kaggle metric.

v10 Primary-source verification audit

Scope. Every source-dependent mechanism and headline claim in the 12 solver sections was re-checked against primary papers, official repositories/code, released traces/configs, and the ARC Prize/Kaggle benchmark surfaces where applicable. Qwen/NVARC design choices that are not literal source behavior are now explicitly labeled LOCAL ADAPTATION or LOCAL INFERENCE. This appendix records the verification boundary so future notebook changes can cite one stable audit.

Legend: [VERIFIED] source/code supports the statement; [CAVEAT] source supports it with an important qualification; [LOCAL] our portability adaptation/inference; [CORRECTED] v9 wording was too strong or inaccurate and is fixed in v10.

Solver	Audit	Result / regime	v10 conclusion
1. Confluence Labs	[CAVEAT]	97.92% Public Eval self-report	Core loop verified. Per-transform child timeout and branch-level JSON persistence are our hardening; prompt contains a short generic ARC heuristic list.
2. Squeeze-Evolve	[VERIFIED]	97.5% ARC-AGI-V2 Public Eval	Paper config and diversity search verified. Generic Python routing is indeed reversed for diversity; later plugin corrects direction.
3. Athanor	[CAVEAT]	113.83/119 = 95.7% Public Eval	One public task excluded; released score is selected development upper-envelope, not one uniform rerun.
4. Imbue Darwinian Evolver	[VERIFIED]	95.1% Public Eval, Gemini 3.1 Pro	Population evolution, 0.90/0.07/0.03 fitness, weighted sampling, novelty and changed-output mutation gate verified.
5. Noemon	[CAVEAT]	92.5% public self-report	Repo says Public Eval; current ARC Prize leaderboard says Public Train. Train-blind induction / LOO validator mechanics verified.
6. ARCgentica	[VERIFIED]	85.28% Public Eval, Opus 4.6	Subagents, executable transforms, subprocess timeout and pass@2 behavior verified; source selection really can keep train-failing attempts.
7. PiVOTools	[VERIFIED]	Public same-model deltas	Stateful REPL, large same-model coding lift and reasoning-state

			fragility verified. Our fresh exact gate is intentionally stronger.
8. Poetiq	[VERIFIED]	54.0% Semi-Private - ARC Prize verified	Fresh-call code iteration, deterministic feedback, released max_iterations=10/max_solutions=5 and multi-expert diversity verified.
9. Johan Land	[CAVEAT]	72.9% Semi-Private; 76.1% Public	Oracle/selection gap verified. +7 vs majority is post-hoc on same candidate pool; 3-vs-1 judge gain is not isolated.
10. CoreThink	[VERIFIED]	24.4% Public; ensemble 30.8%	Object/DSL/cross-example verification confirmed. 30.8 meta-system only selects from 2 CoreThink + 2 ARC Lang candidates; no new generation.
11. Jeremy Berman ARC Lang	[CAVEAT]	29.4% Semi-Private - ARC Prize verified	LOO instruction scoring/revision verified. Follower is allowed to override flawed instruction parts, so score is not pure instruction quality.
12. Eric Pang	[CORRECTED]	26.0% Semi-Private - ARC Prize verified	Library/retrieval verified. v9 incorrectly implied only solved/near-solved programs are stored; source stores the best generated program by train scoring.

1. Confluence Labs - program synthesis + exact verifier

- **[VERIFIED]** Agent prompt requires a Python `transform(grid)` artifact, all-training-pair testing, repeated repair, and execution-derived final grids rather than prose transcription.
- **[VERIFIED]** `test_transform()` loads `transform.py`, checks NumPy array output and exact equality on every training pair; failed iterations resume the latest Gemini session with concise verifier feedback.
- **[VERIFIED]** Workspace is built per test input; assistant prose is deliberately ignored for authoritative grid extraction in favor of explicit output files/tool-result execution.
- **[CAVEAT]** The prompt is compact but includes five generic ARC hints: object manipulation, color changes, spatial patterns, object relationships, and borders/separators/subgrids.
- **[LOCAL]** Fresh child-process/process-group timeout per transform and immediate per-branch JSON checkpointing are our Kaggle safety additions, not exact Confluence implementation details.

Primary evidence: https://github.com/confluence-labs/arc-agi-2 | https://github.com/confluence-labs/arc-agi-2/blob/main/gemini-cli-solver/agent_runner.py | https://github.com/confluence-labs/arc-agi-2/blob/main/submission.py

2. Squeeze-Evolve - diversity-driven evolutionary recombination

- **[VERIFIED]** The ARC-AGI-V2 paper setup uses Gemini 3.1 Pro High as ancestor with N=4, K=2 and T=10, reporting 97.5% public-eval accuracy.
- **[VERIFIED]** Python operators define diversity as number of unique final answers and support uniform K-subset grouping, percentile routing, majority/random Lite aggregation, and replace/accumulate population updates.
- **[VERIFIED]** Current Python orchestrator still passes diversity fitness through generic `assign_routes()`, whose monotonic direction treats high fitness as easier/cheaper; that is reversed when fitness means disagreement.
- **[VERIFIED]** The later official plugin explicitly documents the corrected diversity direction: consensus -> free medoid; higher disagreement -> stronger tier.
- **[LOCAL]** Grid canonicalization, same-Qwen OFF/THINK/XHIGH capability tiers, and K=2 -> binary-consensus interpretation are our ARC/Qwen adaptations.

Primary evidence: https://arxiv.org/abs/2604.07725 | https://github.com/squeeze-evolve/squeeze-evolve/blob/main/src/squeeze_evolve/algorithm/operators.py | https://github.com/squeeze-evolve/squeeze-evolve/blob/main/src/squeeze_evolve/algorithm/orchestrator.py | https://github.com/squeeze-evolve/squeeze-evolve/blob/main/plugins/opencode-squeeze-evolve/src/core.ts

3. Athanor - executable experiments + independent reflection

- **[VERIFIED]** Released audit reports 113.83/119 = 95.7% on ARC-AGI-2 public eval; `b6f77b65` is excluded because of a documented data-quality issue.
- **[VERIFIED]** The core architecture separates exploratory code from final solution code, uses executable verification, an independent artifact-only reflector, compression/reflection across bounded context, and can return APPROVE / REJECT / EXPAND_CANDIDATES.
- **[CAVEAT]** The 119 released checkpoints are selected from development runs: best score per puzzle, lowest cost on ties; 89/119 use the final prompt and 30 earlier drafts. It is an upper envelope, not a single frozen-seed rerun.
- **[LOCAL]** Our subprocess sandbox, fixed 65-75% context compaction threshold, and sequential same-Qwen reviewer are portability hardening choices.
- **[LOCAL]** Treat claims about uniqueness relative to all other top systems as author interpretation unless independently reproduced; keep the actual mechanism evidence separate from comparative rhetoric.

Primary evidence: https://github.com/dastin359/athanor | https://github.com/dastin359/athanor/blob/main/RESULTS.md | https://github.com/dastin359/athanor/blob/main/src/athanor/solver/independent_reflector.py

4. Imbue Darwinian Evolver - population evolution over ARC programs

- **[VERIFIED]** Imbue reports 95.1% public eval with Gemini 3.1 Pro and up to 16 evolution iterations.

- **[VERIFIED]** ARC fitness is `0.90 * correctness + 0.07 * transfer + 0.03 * simplicity`; correctness includes soft cell-level progress plus a full-correctness bonus.
- **[VERIFIED]** Weighted parent sampling combines sigmoid performance with a novelty bonus based on how many children a parent already has; the released ARC run uses sharpness 10, midpoint percentile 99, novelty weight 0.2 and batch size 32.
- **[VERIFIED]** ARC mutation verification only requires a child to produce a different prediction from its parent on at least one input; it is a changed-behavior gate, not proof of improvement.
- **[LOCAL]** Our first Qwen port intentionally drops expensive transfer/simplicity judges and crossover until a small correctness-driven archive proves useful.

Primary evidence: https://imbue.com/blog/2026-02-27-arc-agi-2-evolution | https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/problems/arc_agi.py | https://github.com/imbue-ai/darwinian_evolver/blob/1a50d6995e893580c4bd5071725eb24f2317d380/darwinian_evolver/population.py

5. Noemon - test-blind rule induction + fresh-context validation

- **[VERIFIED]** Round 1 serializes training pairs without the test input; round 2 feeds validation evidence/additional training pairs; round 3 reveals the test input.
- **[VERIFIED]** Validator applies supplied instructions to a held-out training input while its output is hidden and other train pairs remain visible, creating a leave-one-out rule-generalization check.
- **[VERIFIED]** State contains checked/solved indices, structured feedback, duplicate-candidate detection, final-solver candidates and a judge stage.
- **[CAVEAT]** Source state can accumulate solved training indices across revisions; if instructions change, “all indices solved at some point” is not a proof that the current rule solves every train pair. Our final full current-rule sweep fixes this.
- **[CAVEAT]** Result regime is inconsistent across primary surfaces: repository says 92.5% Public Eval, while the current ARC Prize Community Leaderboard labels the same entry Public Train. Do not present it as verified semi-private.
- **[LOCAL]** Cache invalidation by rule hash and a mandatory final all-train revalidation are our hardening, not source-exact behavior.

Primary evidence: https://github.com/noemon/noemon-arc-agi | https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/prompts.py | https://github.com/noemon/noemon-arc-agi/blob/54c7c3af9aa95a495586cba8caffc9ca526f9924/src/arc_agi_benchmarking/noemon/engine.py | https://arcprize.org/leaderboard/community

6. ARCgentica - persistent REPL + scoped sub-agents

- **[VERIFIED]** Repository reports 85.28% ARC-AGI-2 public eval with Opus 4.6 at about $6.94/task; defaults allow up to 10 sub-agents and two independent attempts.
- **[VERIFIED]** Parent/sub-agents can use a Python REPL with exact `accuracy()` and `soft_accuracy()` helpers, propose executable transforms, and the harness runs final code in a fresh temp-directory subprocess with a hard wall timeout.
- **[VERIFIED]** Attempt harness computes train soft score but still saves test outputs even when a training example is not exact.
- **[VERIFIED]** `score.py` selects the first two non-empty test grids by attempt order. There is no exact-train-fit filter, dedup, or learned judge in that selection path.

- **[LOCAL]** Our port keeps dynamic delegation only after a parent is train-exact and ambiguity remains, with a hard child budget; this is deliberately stricter than source selection.

Primary evidence: https://github.com/symbolica-ai/arcgentica/blob/main/README.md | https://github.com/symbolica-ai/arcgentica/blob/main/solve.py | https://github.com/symbolica-ai/arcgentica/blob/main/score.py | https://github.com/symbolica-ai/arcgentica/blob/main/arc_agent/agent.py

7. PiVOTools - same-model stateful Python interleaving

- **[VERIFIED]** Public results show large same-model gains from agentic Python: GPT-OSS-120B High 6.11% -> 26.38% and MiniMax M2.1 3.06% -> 10.56% on full public 120; GPT-5.2 XHigh 59.81% -> 73.36% on the common 107-task slice.
- **[VERIFIED]** The implementation uses a stateful IPython/Python REPL while model-side reasoning/tool state must be manually preserved across requests; the authors explicitly document inference-stack soft failures when that state is lost.
- **[VERIFIED]** Generated programs are independently checked on training examples by utility code and executed on test inputs; independent samples are combined into pass@2 candidates.
- **[LOCAL]** Our PIV-Q27-P0 forces a fresh all-train exact gate before test execution and explicitly compares Qwen preserved-thinking versus compact-state policies. That stronger gate/history policy is ours, not a PiV source claim.

Primary evidence: https://github.com/gutfeeling/arc-agi-2-submission | https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/solver/solver.py | https://github.com/gutfeeling/arc-agi-2-submission/blob/master/src/arcagi2/tools/repl_tool.py | https://huggingface.co/datasets/arcagi2/arcagi2-agentic-coding-publication

8. Poetiq - fresh-call feedback-guided program search

- **[VERIFIED]** ARC Prize verified the open-source pure-Gemini configuration at 54.0% on ARC-AGI-2 Semi-Private.
- **[VERIFIED]** Each iteration is a fresh model call over the re-serialized puzzle plus selected previous program/feedback examples; train failures generate exact success labels, shape mismatch, cell diff, runtime errors and soft scores.
- **[VERIFIED]** Released default uses `max_iterations=10`, `max_solutions=5`, `selection_probability=1.0`, shuffled examples, improving-order feedback, and best-result fallback.
- **[VERIFIED]** If one program is exact on all training examples, the solver returns immediately with its test executions.
- **[LOCAL]** PTQ-Q27-T0 deliberately starts with much smaller iteration/history budgets so PIV-vs-PTQ measures statefulness rather than merely more compute.

Primary evidence: https://poetiq.ai/posts/arcagi_verified/ | https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/solve_coding.py | https://github.com/poetiq-ai/poetiq-arc-agi-solver/blob/main/arc_agi/config.py | https://arcprize.org/leaderboard/community

9. Johan Land - candidate-family coverage + holistic trace judging

- **[VERIFIED]** Paper reports 72.9% ARC-AGI-2 Semi-Private and 76.1% task-level Public; on 167 public test instances it reports 144/167 = 86.23% candidate oracle and 128/167 = 76.65% selected pass@2.

- **[VERIFIED]** Failure accounting distinguishes 22 generation failures from 17 selection failures, making oracle-gap analysis a first-class diagnostic.
- **[VERIFIED]** Generation families are staged and heterogeneous (text/image/code), candidate provenance/traces are retained, and the holistic selector uses three parallel GPT-5.2 judges over the candidate evidence.
- **[CAVEAT]** The +7 solved instances versus majority are a post-hoc same-pool selector analysis with synthesis disabled in both comparison arms, not a newly rerun end-to-end generation experiment; synthesis adds +1.
- **[CAVEAT]** The paper does not provide a clean one-judge versus three-judge causal ablation. Our JUD-Q27-J0 should therefore start with one judge only after candidate coverage improves.

Primary evidence: https://arxiv.org/abs/2606.31543 | https://github.com/beetree/ARC-AGI/tree/Johan_Land_Solver_V7 | https://github.com/beetree/ARC-AGI/blob/Johan_Land_Solver_V7/src/selection_advanced.py

10. CoreThink - object-centric neural-symbolic cross-example verification

- **[VERIFIED]** CoreThink combines object extraction/representation, neural mechanism hypotheses over a fixed DSL, and cross-example symbolic consistency checks; standalone public result is about 24.4%.
- **[VERIFIED]** Ablation in the paper separates the gains from symbolic hints and cross-example consistency, supporting them as distinct components rather than one opaque prompt trick.
- **[VERIFIED]** The reported ~30.8% meta-system uses a four-candidate pool (two CoreThink + two ARC Lang) and selects twice without replacement; the selector does not generate new candidates.
- **[LOCAL]** Our CORE-Q27-C0 should therefore be judged as a complementary specialist branch by NVARC+Qwen oracle-union additions, not by whether it beats the general Qwen branch standalone.
- **[LOCAL]** NumPy-first object extraction and any reduced mechanic vocabulary are implementation choices; the paper's exact DSL should not be copied blindly.

Primary evidence: https://arxiv.org/abs/2604.02434 | https://github.com/CoreThink-AI/arc-agi-2-reasoner | https://github.com/CoreThink-AI/arc-agi-2-reasoner/blob/main/arc_agi/src/patterns/find_patterns.py

11. Jeremy Berman / ARC Lang - evolve natural-language instructions

- **[VERIFIED]** ARC Prize Community Leaderboard records 29.4% on ARC-AGI-2 Semi-Private; ARC Prize states semi-private ARC-AGI-1/2 entries are run and verified by ARC Prize.
- **[VERIFIED]** Candidate natural-language instructions are scored with leave-one-out execution: one training pair becomes the held-out target, its output is hidden, and another LLM follows the instructions using the remaining training examples.
- **[VERIFIED]** Revision uses concrete incorrect outputs/diffs; pooling synthesizes a new instruction set from multiple scored instructions; final stage can produce up to two diverse guesses.
- **[CAVEAT]** The follower prompt explicitly allows ignoring erroneous parts of the instructions when necessary to obtain the correct grid. Consequently, a high instruction score is partly a property of the follower's reasoning capability.
- **[LOCAL]** LANG-Q27-L0 will constrain the follower to rule execution/local ambiguity repair so we can measure instruction quality more cleanly.

Primary evidence: https://arcprize.org/leaderboard/community | https://github.com/jerber/arc-lang-public | https://github.com/jerber/arc-lang-public/blob/main/src/run.py | https://github.com/jerber/arc-lang-public/blob/main/src/main.py

12. Eric Pang - frozen reusable program library

- **[VERIFIED]** ARC Prize Community Leaderboard records 26.0% ARC-AGI-2 Semi-Private, verified under ARC Prize's semi-private policy.
- **[VERIFIED]** Source builds a cross-task library of Python primitives; retrieval executes every primitive on the current training examples, ranks first by number of exactly correct examples and then by mean cell-level accuracy, and can use score-weighted sampling.
- **[VERIFIED]** The write-up reports seeding from the 1,000 public-training tasks, producing 538 stored programs after invalid/time-out generations, then two semi-private rounds with five programs per task.
- **[CORRECTED]** Library insertion is the best newly generated program by the source's primary/secondary train scores. It does not require the program to be solved/train-perfect/near-solved.
- **[LOCAL]** For competition hygiene our LIB-Q27-E0 build phase ends before eval/private inference; the resulting public-training library is frozen and read-only at inference time.

Primary evidence: https://arcprize.org/leaderboard/community | https://github.com/epang080516/arc_agi | https://github.com/epang080516/arc_agi/blob/main/src/logic.py | https://github.com/epang080516/arc_agi/blob/main/src/submission.py

New findings added after the v9 audit

- **Official 2026 scoring denominator:** Kaggle scores each task test output separately: exactly two predictions (`attempt_1`, `attempt_2`), either exact match earns 1, and the final score averages across all task test outputs. Because historical papers/repos use mixed task-level/task-normalized reporting, every future table must label its denominator. Primary: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/overview/description
- **Noemon regime-label conflict:** The current ARC Prize Community Leaderboard labels Noemon 92.5% as ARC-AGI-2 Public Train, while its repository says Public Eval. Keep the split label unresolved unless the authors/ARC Prize reconcile it. Primary: https://arcprize.org/leaderboard/community
- **ArunSehrawat public-training program corpus:** `ArunSehrawat/arc-agi2-solutions` (2026-08-15) provides verified executable programs for 383 unique ARC-AGI-2 public-training task IDs: 222 manual, 158 BARC-converted and 37 LLM-assisted, with 34 overlaps. This can cheaply bootstrap LIB-Q27-E0 after our own re-verification. Use only public-training IDs; keep BARC-derived files in a separate licensing bucket because the upstream BARC code has no published LICENSE. Primary: https://github.com/ArunSehrawat/arc-agi2-solutions
- **ARCANA watchlist only:** ARCANA (arXiv:2607.09059, 2026-07-10) proposes object-centric scene graphs, a latent DSL program policy, symbolic execution, reflection, a shared differentiable blackboard and learned scheduling. The abstract is architecturally relevant, but this audit did not find a strong reproducible ARC-AGI-2 score/code artifact sufficient to displace our controlled PIV/PTQ experiments. Primary: https://arxiv.org/abs/2607.09059

- **Independent support for "coverage before ranking":** A July-2026 ARC-AGI-1 study (`arXiv:2607.06764`) reports its candidate selection captures about 95% of the candidate ceiling and diagnoses the system as generation-bound; adaptive re-exploration then improves pass@2. It is ARC-AGI-1, not directly comparable to ARC-AGI-2, so it is supporting evidence only - but it reinforces our oracle-gap diagnostic and judge-later ordering. Primary: https://arxiv.org/abs/2607.06764
- **Manual DSL repositories are references, not solvers:** StAy-KG's manual ARC-AGI-2 DSL reports 56/59 hand-solved task-specific programs and explicitly states they do not generalize to unseen IDs. Use such repos only for primitive vocabulary, regression fixtures, or failure taxonomy; never import task-specific eval/private solutions into the competition solver. Primary: https://github.com/StAy-KG/arc-agi-2-dsl

Impact on our Qwen3.8-27B + NVARC experiment order

Decision: No newly verified source changes the priority order. The new evidence mostly tightens evaluation hygiene and reduces bootstrap cost for later branches.

- 0. Finish Qwen3.8-27B TP=4 load/warmup proof and retain direct OFF/XHIGH baselines.
- 1. Run PIV-Q27-P0 (stateful REPL) versus PTQ-Q27-T0 (fresh-call deterministic feedback) on the same frozen proxy subset and matched generation budget.
- 2. Record both current Kaggle output-weighted pass@2 and task-normalized/full-task metrics, plus NVARC union/oracle, unique GT additions, train-perfect rate, tokens and wall-clock.
- 3. If agentic code search adds complementary coverage, run LANG-Q27-L0. Only then test CORE-Q27-C0 and LIB-Q27-E0; the new verified public-training program corpus can bootstrap LIB-Q27 but must be frozen before eval/private inference.
- 4. Add JUD-Q27-J0 only when oracle coverage materially exceeds selected pass@2. Do not pay selector compute to compensate for missing candidates.
- 5. Keep Squeeze/Imbue/Athanor/ARCgentica-style heavier evolution/delegation as later ablations after the low-risk PIV/PTQ mechanisms are measured.

v10 source-control rule: When a statement is copied from a paper/repository it must preserve the original evaluation split and denominator. When we change timeout strategy, verifier strictness, prompt scope, state compaction, library freezing, judge count, or routing logic, the note must call it LOCAL rather than silently presenting it as source behavior.